

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC THĂNG LONG



KHOÁ LUẬN TỐT NGHIỆP

NGHIÊN CỨU VÀ XÂY DỰNG MÔ HÌNH NHẬN DIỆN DEEPPAKE (CÔNG NGHỆ GIẢ MẠO KHUÔN MẶT)

SINH VIÊN THỰC HIỆN:	ĐỖ VIỆT ANH
MÃ SINH VIÊN:	A45801
KHÓA HỌC:	K35 – TRÍ TUỆ NHÂN TẠO
GV HƯỚNG DẪN:	PGS.TS. NGUYỄN HOÀNG PHƯƠNG

HÀ NỘI - 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC THĂNG LONG



KHOÁ LUẬN TỐT NGHIỆP

NGHIÊN CỨU VÀ XÂY DỰNG MÔ HÌNH NHẬN DIỆN DEEPPAKE (CÔNG NGHỆ GIẢ MẠO KHUÔN MẶT)

SINH VIÊN THỰC HIỆN: ĐỖ VIỆT ANH

MÃ SINH VIÊN: A45801

KHÓA HỌC: K35 – TRÍ TUỆ NHÂN TẠO

GV HƯỚNG DẪN: PGS.TS. NGUYỄN HOÀNG PHƯƠNG

HÀ NỘI – 2026

LỜI CẢM ƠN

Lời đầu tiên tôi xin được phép gửi lời cảm ơn chân thành nhất tới toàn bộ các thầy cô giáo trong khoa Toán - Tin cũng như các thầy cô giảng dạy trong trường Đại học Thăng Long đã truyền đạt những kiến thức quý báu và bổ ích nhất cho tôi trong những năm học vừa qua.

Đặc biệt, tôi xin chân thành cảm ơn PGS.TS. Nguyễn Hoàng Phương đã tận tình hướng dẫn, động viên và trực tiếp giúp đỡ trong suốt quá trình thực hiện khóa luận tốt nghiệp.

Để có được kết quả như ngày hôm nay, tôi rất biết ơn các thầy cô trường Đại học Thăng Long, gia đình và bạn bè đã động viên, khích lệ, tạo mọi điều kiện thuận lợi nhất cho tôi trong suốt quá trình học tập cũng như quá trình thực hiện khóa luận tốt nghiệp này.

Vì kiến thức còn hạn chế, trong quá trình triển khai và thực hiện khóa luận tốt nghiệp này tôi sẽ có những sai sót không mong muốn. Tôi rất mong nhận được những ý kiến đóng góp từ các thầy cô để bổ sung thêm kiến thức và hoàn thiện khóa luận tốt nghiệp này tốt hơn.

SINH VIÊN

Chữ ký

Đỗ Việt Anh

LỜI CAM ĐOAN

Tôi xin cam đoan đề tài này được thực hiện dựa trên lý thuyết, thuật toán xử lý dữ liệu, thu thập dữ liệu và các mô hình học máy được trình bày trong khoá luận là do tôi thực hiện dưới sự hướng dẫn của PGS.TS. Nguyễn Hoàng Phương. Mọi tài liệu, các bài báo và công cụ của các tác giả khác được sử dụng đều có trích dẫn tường minh về nguồn gốc và tác giả trong danh sách tài liệu tham khảo. Tôi xin chịu hoàn toàn trách nhiệm về lời cam đoan này!

Hà Nội, ngày 10 tháng 01 năm 2026

SINH VIÊN

Chữ ký

Đỗ Việt Anh

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU CHUNG	1
1.1. Đặt vấn đề	1
1.2. Khái niệm về deepfake	1
1.3. Mục tiêu	1
1.4. Phương pháp nghiên cứu	2
1.5. Đối tượng nghiên cứu	2
1.6. Phạm vi nghiên cứu	2
1.7. Bố cục khóa luận	3
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	4
2.1. PyTorch	4
2.2. Transformers	4
2.3. Xception	7
2.3.1. Luồng vào (Entry flow)	8
2.3.2. Luồng giữa (Middle flow)	8
2.3.3. Luồng thoát (Exit flow)	9
2.4. MTCNN	9
2.4.1. P-Net (Proposal Network)	10
2.4.2. R-Net (Refine Network)	11
2.4.3. O-Net (Output Network)	11
2.5. Long Short-Term Memory (LSTM)	12
2.5.1. Cổng quên (Forget Gate)	13
2.5.2. Cổng vào (Input Gate)	13
2.5.3. Cập nhật cell state (Memory Highway)	14
2.5.4. Cổng đầu ra (Output Gate)	14
2.6. EfficientNet	15
CHƯƠNG 3. TIỀN XỬ LÝ DỮ LIỆU	18
3.1. Tổng quan về dữ liệu	18

3.2. Tiền xử lý dữ liệu	19
3.2.1. Chia dữ liệu.....	19
3.2.2. Chia lại nhãn	19
3.2.3. Trích xuất khung hình	20
3.2.4. Phát hiện và cắt khuôn mặt.....	20
3.2.5. Căn chỉnh khuôn mặt	20
3.2.6. Tăng cường dữ liệu	21
CHƯƠNG 4. XÂY DỰNG VÀ HUẤN LUYỆN MÔ HÌNH	24
4.1. Xây dựng mô hình EfficientNet kết hợp Temporal Attention	24
4.1.1. Trích xuất đặc trưng không gian bằng EfficientNet	25
4.1.2. Temporal Attention (Cơ chế chú ý theo thời gian)	25
4.1.3. Phân loại và đưa ra quyết định	26
4.2. Xây dựng mô hình Xception kết hợp mạng LSTM.....	27
4.2.1. Trích xuất đặc trưng không gian bằng Xception	27
4.2.2. Phân tích chuỗi thời gian bằng LSTM.....	28
4.2.3. Phân loại và đưa ra quyết định	28
4.3. Xây dựng mô hình Xception kết hợp Transformer Encoder	28
4.3.1. Trích xuất đặc trưng không gian bằng Xception	29
4.3.2. Mô hình hóa quan hệ thời gian bằng Transformer Encoder.....	29
4.3.3. Tổng kết đặc trưng và phân loại.....	30
4.4. Huấn luyện mô hình	30
4.4.1. Trọng số cân bằng dữ liệu	30
4.4.2. Các siêu tham số huấn luyện mô hình	30
CHƯƠNG 5. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ.....	32
5.1. Chỉ số đánh giá	32
5.2. So sánh các mô hình	32
5.3. Nhận xét và đánh giá kết quả của mô hình.....	32
5.4. Phân tích ma trận nhầm lẫn	33

5.5. Thảo luận và ý nghĩa thực tiễn.....	34
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	36
6.1. Kết luận	36
6.2. Hướng phát triển	36
CHƯƠNG 7. DEMO ỨNG DỤNG.....	37
TÀI LIỆU THAM KHẢO.....	38

DANH MỤC VIẾT TẮT

Viết tắt	Tên đầy đủ
AI	Artificial Intelligence
GPU	Graphics Processing Unit
GANs	Generative Adversarial Networks
ML	Machine Learning
NLP	Natural Language Processing
CNN	Convolutional Neural Network
MTCNN	Multi-task Cascaded Convolutional Networks
RGB	Red, Green, Blue
AUC-ROC	Area Under the Receiver Operating Characteristic curve
ELA	Error Level Analysis

DANH MỤC HÌNH ẢNH

Ảnh 2.1. Khái niệm về kiến trúc Xception	7
Ảnh 2.2. Kiến trúc của Xception	8
Ảnh 2.3. Kiến trúc của MTCNN	10
Ảnh 2.4. Kiến trúc của LSTM	12
Ảnh 2.5. Kiến trúc mô hình EfficientNetB0	16
Ảnh 3.1. Bộ dữ liệu FaceForensics++	18
Ảnh 3.2. Phát hiện 5 điểm landmark trên khuôn mặt bằng mô hình MTCNN	21
Ảnh 3.3. Ví dụ ảnh đã được tăng cường	22
Ảnh 4.1. Kiến trúc mô hình EfficientNet +Temporal Attention	24
Ảnh 4.2. Kiến trúc mô hình Xception + LSTM.....	27
Ảnh 4.3. Kiến trúc mô hình Xception + Transformer Encoder	29
Ảnh 5.1. Ma trận nhầm lẫn	34
Ảnh 7.1. Demo ứng dụng.....	37

DANH MỤC CÁC BẢNG

Bảng 5.1. So sánh kết quả giữa các mô hình	32
Bảng 5.2 Kết quả thực nghiệm	33

Thuật ngữ trong công nghệ thông tin	
Thuật ngữ	Ý nghĩa
Deepfake	Kỹ thuật sử dụng mạng học sâu để lồng ghép khuôn mặt của một chủ thể vào video nguồn nhằm tạo ra nội dung giả mạo
Trí tuệ nhân tạo	Trí tuệ do con người lập trình tạo nên với mục tiêu giúp máy tính có thể tự động hóa các hành vi thông minh như con người.
Dữ liệu	Một tập hợp các dữ kiện, chẳng hạn như số, từ, hình ảnh, nhằm đo lường, quan sát hoặc chỉ là mô tả về sự vật
Pos_weight	Trọng số được sử dụng để cân bằng tầm quan trọng giữa hai lớp khi tập dữ liệu bị mất cân bằng
Siêu tham số	Những giá trị tham số có thể điều chỉnh quá trình huấn luyện của mô hình
Tập train	Tập dữ liệu sử dụng để huấn luyện mô hình
Tập Vadiation	Tập dữ liệu dùng để đánh giá mô hình trong quá trình đang huấn luyện.
Tập test	Tập dữ liệu sử dụng để kiểm tra độ chính xác của mô hình
Overfitting	Hiện tượng quá khớp, khi mô hình học quá kỹ trên tập huấn luyện dẫn đến khả năng tổng quát hóa kém trên dữ liệu mới
Epoch	Một chu kì hoàn chỉnh, trong đó toàn bộ dữ liệu huấn luyện được đưa qua mạng nơ-ron một lần
Loss	Giá trị đo lường sai số giữa dự đoán của mô hình và nhãn thực tế
Batch size	Số lượng mẫu dữ liệu được đưa vào mô hình để tính toán trong một lần lặp trước khi cập nhật trọng số.
Learning rate	Một siêu tham số kiểm soát mức độ thay đổi của trọng số mô hình trong mỗi lần cập nhật dựa trên sai số tính được

CHƯƠNG 1. GIỚI THIỆU CHUNG

1.1. Đặt vấn đề

Hiện nay, với sự phát triển mạnh của các mô hình trí tuệ nhận tạo, nó đem tới cho nhân loại một sự phát triển vượt bậc cả về mặt tích cực và tiêu cực. Trong đó, Deepfake (công nghệ giả mạo khuôn mặt và giọng nói) đã trở thành một trong những công nghệ đóng góp sự phát triển vượt bậc của AI tới xã hội hiện nay.

Ở khía cạnh tích cực, Deepfake giúp ích nhiều trong các lĩnh vực như: điện ảnh, làm phim, game, bảo tồn di sản văn hóa,... Tuy nhiên trong thực tế hầu như deepfake bị lạm dụng như một công cụ phạm pháp, giả mạo khuôn mặt, giọng nói,... của người khác nhằm thực hiện các hành vi lừa đảo, đặc biệt nó còn bị đem làm công cụ để đăng tin giả nhằm thao túng dư luận.

Theo báo cáo năm 2024 của Global Initiative Against Transnational Organized Crime chỉ ra từ giữa năm 2022 đến 2023, khu vực Châu Á – Thái Bình Dương tăng mạnh các vụ lừa đảo liên quan đến deepfake chỉ sau Bắc Mỹ. Trong đó đáng chú ý, Việt Nam là nước có mức tăng về gian lận deepfake cao nhất khu vực với tỷ lệ 25,3%, đứng thứ 2 là Nhật Bản với mức tăng là 23,4% [2].

Trong những năm gần đây, deepfake càng phát triển nhanh chóng nhờ các yếu tố: sự phát triển của các mô hình học sâu, GPU dần phổ biến hơn, và do sự phát triển của mạng xã hội tạo ra nhiều bộ dữ liệu có sẵn để phục vụ cho huấn luyện. Nó đem tới nhiều thách thức đối với pháp luật và xã hội. Vì vậy việc nghiên cứu và xây dựng mô hình phát hiện deepfake mang tính cấp bách nhằm ngăn chặn mối nguy cơ do công nghệ này mang lại.

1.2. Khái niệm về deepfake

Deepfake là sự kết hợp hai thuật ngữ của mô hình học sâu “deep learning” và cơ chế giả tạo “fake” đây là kỹ thuật sử dụng mạng học sâu GANs [3] và gần đây là Diffusion Models lồng ghép khuôn mặt của một chủ thể mục tiêu vào video nguồn để tạo video của chủ thể mục tiêu với cử chỉ của video nguồn. Từ đó, có thể giả mạo bất kì hành vi nào của chủ thể mục tiêu.

1.3. Mục tiêu

Trong dự án này, nhóm tập trung vào nghiên cứu và xây mô hình nhận diện deepfake nhằm phân biệt video thật và giả, góp phần ngăn chặn các hành vi lừa đảo và tin giả. Các mục tiêu cụ thể bao gồm:

- Tìm hiểu và thu thập dữ liệu: Nghiên cứu bộ dữ liệu FaceForensics++ bao gồm các kỹ thuật giả mạo phổ biến như NeuralTextures, Face2Face, FaceSwap, Deepfakes,..[4]
- Xử lý dữ liệu: Nghiên cứu các phương pháp trích xuất khung hình, phát hiện khuôn mặt bằng MTCNN, căn chỉnh khuôn mặt và tăng cường dữ liệu.
- Xây dựng mô hình: Thiết lập mạng nơ-ron kết hợp giữa mạng CNN để trích xuất đặc trưng không gian và các mô hình học sâu để mô hình hóa quan hệ thời gian.
- Huấn luyện và tối ưu: Thực hiện tinh chỉnh các siêu tham số và trọng số lớp để cải thiện hiệu suất.
- Đánh giá thực nghiệm: Phân tích kết quả thông qua các chỉ số Accuracy, AUC-ROC và ma trận nhầm lẫn (Confusion Matrix).
- Xây dựng ứng dụng: Cài đặt ứng dụng Demo hỗ trợ nhận diện trực tiếp trên màn hình thiết bị.

1.4. Phương pháp nghiên cứu

Khóa luận sử dụng hai phương pháp tiếp cận chính:

- Phương pháp tổng hợp tài liệu: Nghiên cứu các bài báo khoa học về Deepfake, và các kỹ thuật xử lý đối với bài toán nhận diện deepfake.
- Phương pháp thực nghiệm: Xây dựng mô hình trên PyTorch, tiến hành huấn luyện trên tập dữ liệu lớn và thử nghiệm thực tế qua ứng dụng Demo.

1.5. Đối tượng nghiên cứu

- Bài toán nghiên cứu: Nhận diện và phân loại video Deepfake dựa trên sự bất thường về không gian (khuôn mặt) và thời gian (sự nhất quán giữa các khung hình).
- Dữ liệu sử dụng: Bộ dữ liệu FaceForensics++ gồm 1.000 video gốc và các video được tạo ra từ 5 kỹ thuật giả mạo khác nhau.

1.6. Phạm vi nghiên cứu

- Dữ liệu: Giới hạn trong tập dữ liệu FaceForensics++ với các kỹ thuật Deepfake, Face2Face, FaceSwap, FaceShifter, và NeuralTextures,..
- Kiến trúc mô hình: Tập trung vào sự kết hợp giữa CNN và các mô hình học sâu.
- Chức năng: Nhận diện video có sự xuất hiện của khuôn mặt người, xác định nhãn nhị phân (Real/Fake).

1.7. Bố cục khóa luận

- Chương 1: Giới thiệu chung: Trình bày đặt vấn đề, mục tiêu của dự án, phương pháp thực hiện, đối tượng, phạm vi nghiên cứu và bố cục của khóa luận.
- Chương 2: Cơ sở lý thuyết: Giới thiệu về thư viện PyTorch, kiến trúc Transformer, mạng Xception, mô hình phát hiện khuôn mặt MTCNN, mạng học tuần tự LSTM và kiến trúc EfficientNet.
- Chương 3: Tiền xử lý dữ liệu: Quy trình chia dữ liệu Train/Test/Valdiation, gộp nhãn nhị phân, trích xuất khung hình từ video, sử dụng MTCNN để cắt/căn chỉnh khuôn mặt và áp dụng các kỹ thuật tăng cường hình ảnh.
- Chương 4: Xây dựng và huấn luyện mô hình: Chi tiết kiến trúc 3 mô hình thử nghiệm bao gồm EfficientNet + Temporal Attention, Xception + LSTM, Xception + Transformer Encoder và thiết lập các siêu tham số huấn luyện.
- Chương 5: Kết quả thực nghiệm và đánh giá: So sánh hiệu suất các mô hình, phân tích ma trận nhầm lẫn và thảo luận về ý nghĩa thực tiễn của dự án này.
- Chương 6: Kết luận và hướng phát triển: Tổng kết ưu nhược điểm và đề xuất cải tiến để nâng cao hiệu suất của mô hình.
- Chương 7: Demo ứng dụng: Giới thiệu phần mềm nhận diện trực tiếp trên màn hình thiết bị sử dụng giao diện CustomTkinter và thư viện mss.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. PyTorch

PyTorch là một thư viện mã nguồn mở dành cho học sâu (Deep Learning), được phát triển bởi Facebook AI Research (FAIR) và công bố vào năm 2016 [5]. PyTorch nhanh chóng trở thành một trong những framework học sâu phổ biến nhất nhờ tính linh hoạt, khả năng biểu diễn mô hình trực quan và hỗ trợ mạnh mẽ cho nghiên cứu lần triển khai thực tế.

Hiện nay, PyTorch nhanh chóng trở thành một trong những framework học sâu phổ biến nhất nhờ tính linh hoạt, khả năng biểu diễn mô hình trực quan và hỗ trợ mạnh mẽ cho nghiên cứu lần triển khai thực tế. Theo báo cáo thường niên năm 2024 của Linux Foundation về xu hướng AI/ML, PyTorch chiếm khoảng 63% thị phần trong không gian "huấn luyện mô hình", vượt xa các đối thủ và trở thành framework được lựa chọn hàng đầu trong cả giới học thuật lẫn công nghiệp [6]. Sự thống trị này một phần đến từ sự tích hợp sâu với Python và hệ sinh thái thư viện phong phú xung quanh nó.

PyTorch có rất nhiều thư viện con hỗ trợ cho đủ loại bài toán khác nhau như:

- Torchvision: Công cụ dành cho xử lý ảnh, cung cấp dataset, mô hình mẫu, và các phép biến đổi ảnh.
- Torchaudio và Torchtext: Hỗ trợ xử lý dữ liệu âm thanh và văn bản.
- PyTorch Lightning, Hydra, Deep Speed: Giúp chuẩn hóa quá trình huấn luyện và tối ưu mô hình lớn.
- ONNX và TorchScript: Hỗ trợ triển khai mô hình sang môi trường sản xuất với hiệu năng cao.

2.2. Transformers

Transformer là một kiến trúc mạng nơ-ron được sử dụng để thực hiện các tác vụ học máy, đặc biệt là trong xử lý ngôn ngữ tự nhiên (NLP) và thị giác máy tính. Mô hình này được Vaswani và cộng sự đề xuất thông qua bài báo "Attention Is All You Need" vào năm 2017 [7].

Cấu trúc Transformer gồm hai phần chính: Encoder và Decoder, mỗi phần được tạo thành từ nhiều layer giống nhau. Trong đó Encoder sẽ biến câu đầu vào thành vector và Decoder sẽ biến vector thành câu đầu ra. Mỗi Encoder và Decoder đều chứa Self-Attention and Feed-Forward Layers.

Encoder bao gồm nhiều lớp (thường là 6 lớp). Mỗi lớp có hai thành phần chính:

- Self-Attention Mechanism: Giúp mô hình hiểu các mối quan hệ từ.

- Feed-Forward Neural Network: Tiếp tục biến đổi làm giàu.

Decoder cũng bao gồm 6 lớp nhưng có thêm cơ chế encoder-decoder attention mechanism. Điều này cho phép bộ giải mã tập trung vào các phần có liên quan của câu đầu vào trong khi tạo đầu ra.

Cơ chế Self-Attention là thành phần cốt lõi của Transformer. Thay vì xử lý tuần tự như RNN, Self-Attention cho phép mỗi token trong chuỗi “nhìn” tất cả các token khác để hiểu mối quan hệ giữa chúng và tính ra mức độ quan trọng của từng cặp token. Cơ chế này hoạt động bằng cách chiếu mỗi token thành ba vector: Query (Q) đại diện cho từ hiện tại cần tìm thông tin, Key (K) đại diện cho những gì mỗi từ có thể đóng góp, và Value (V) là thông tin thực sự của từ đó. Mức độ liên quan giữa token i và token j được tính bằng tích vô hướng giữa Query của i và Key của j , sau đó chuẩn hóa bằng hàm softmax để tạo ra trọng số chú ý. Đầu ra cho token i là tổng trọng số của tất cả các Value. Công thức tổng quát cho một "head" chú ý là:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) \times V \quad (1)$$

Trong đó:

- QK^T : phép nhân ma trận giữa Query và chuyển vị của Key
- d_k : số chiều của vector Key
- V : ma trận Value

Để xác định vị trí chính xác của các token trong chuỗi vì vậy Positional Encoding là cách Transformer bổ sung để Self-Attention có thể xác định vị trí của các token. Mỗi vị trí trong chuỗi được gán một vector đặc biệt có dạng sóng sin-cos, được định nghĩa theo các tần số khác nhau.

$$PE_{(pos, 2i)} = \sin(pos/10000^{\frac{2i}{d_{model}}}) \quad (2)$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{\frac{2i}{d_{model}}}) \quad (3)$$

Trong đó:

- pos : vị trí của token trong chuỗi (bắt đầu từ 0)
- i : chỉ số chiều (dimension index) trong vector mã hóa
- d_{model} : số chiều của biểu diễn token

Cách mã hóa này giúp mô hình phân biệt vị trí tuyệt đối (token đang ở đâu) và cả vị trí tương đối (token cách nhau bao xa). Nhờ đó Transformer nắm được ngữ cảnh theo thứ tự.

Để mở rộng khả năng của mô hình trong việc học các loại quan hệ khác nhau, Transformer còn áp dụng cơ chế Multi-Head Self-Attention, cơ chế này là sự kết hợp của nhiều Self-Attention chạy song song trên cùng 1 chuỗi rồi ghép kết quả lại.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (4)$$

$$\text{where } \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Trong đó:

- h : số lượng head.
- W_i^Q, W_i^K, W_i^V : ma trận trọng số cho head thứ i .
- W^O : ma trận đầu ra.

Cơ chế này cho phép mô hình “nhìn” dữ liệu từ nhiều góc độ cùng lúc, làm biểu diễn giàu thông tin hơn hẳn so với một head đơn lẻ.

Sau Self-Attention, mô hình sử dụng Position-wise Feed-Forward Network, cơ chế này xử lý từng vị trí độc lập (không xét đến các token xung quanh) thông qua phép biến đổi tuyến tính (thường sử dụng hàm kích hoạt ReLU):

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2 \quad (5)$$

Trong đó:

- x : biểu diễn hiện tại của một token sau khi đã qua Attention
- W_1 : ma trận trọng số lớp ẩn
- b_1 : bias cho lớp ẩn
- W_2 : ma trận trọng số đầu ra
- b_2 : bias cho đầu ra

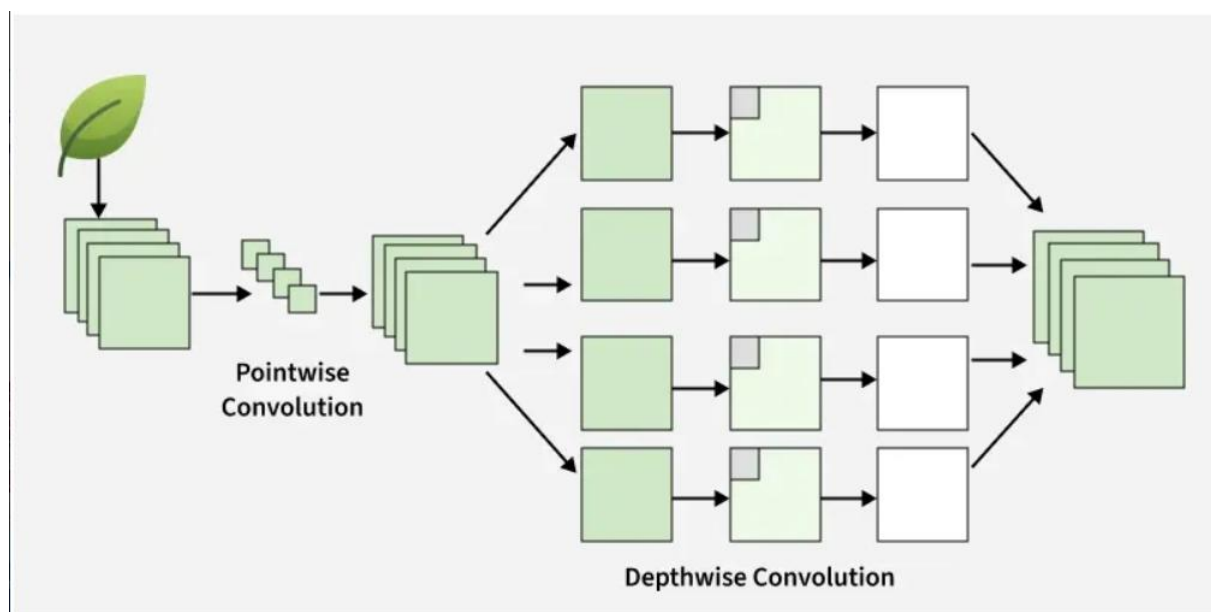
Cơ chế FFN này được áp dụng giống nhau cho tất cả các vị trí trong chuỗi, đóng vai trò biến đổi và làm giàu thêm biểu diễn cục bộ tại mỗi vị trí sau khi đã được tích hợp thông tin toàn cục từ các token khác thông qua Attention. Nó giúp tăng khả năng biểu diễn phi tuyến của mô hình. Kiến trúc kết hợp giữa Attention (nhắm bắt phụ thuộc toàn cục) và FFN (xử lý biểu diễn cục bộ) tạo nên sức mạnh cốt lõi của Transformer.

Trích dẫn : Các công thức (1), (2), (3), (4), (5) được đề xuất trong Vaswani et al. (2017), "*Attention Is All You Need*" [8]

2.3. Xception

Xception là viết tắt của Extreme Inception, là cấu trúc một kiến trúc mạng nơ-ron tích chập sâu (CNN) được xây dựng và cải tiến từ mô hình Inception gốc do François Chollet (2017) đề xuất [9]. Nó thay thế các lớp tích chập tiêu chuẩn bằng các phép tích chập tách chiều sâu (depthwise separable convolutions), Phép tích chập này gồm 2 bước:

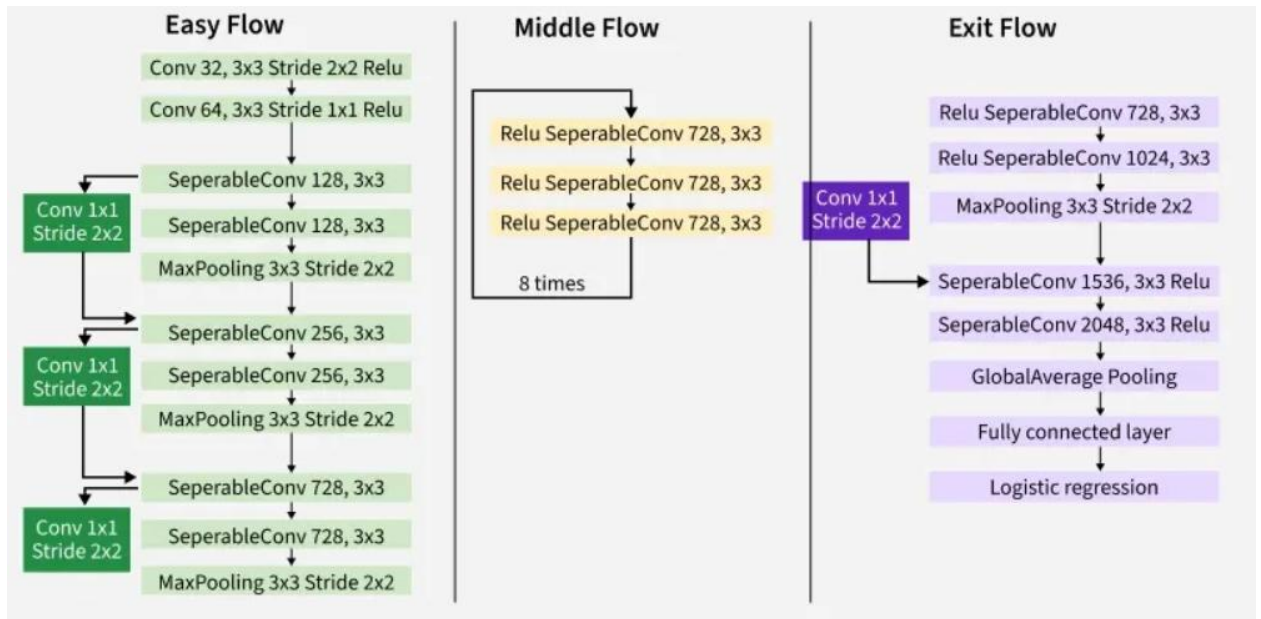
- 1. Depthwise Convolution:
 - + Mỗi kênh đầu vào có bộ lọc riêng và mỗi kênh được xử lý riêng biệt.
 - + Không trộn thông tin giữa các kênh
- 2. Pointwise Convolution (1×1):
 - + Học quan hệ giữa các kênh để kết hợp kết quả từ tất cả các bộ lọc riêng lẻ thành một đầu ra duy nhất.



Ảnh 2.1. Khái niệm về kiến trúc Xception

Các phép tích chập tách chiều sâu này giúp hiệu quả và chính xác hơn cho các tác vụ như phân loại hình ảnh. Sự thay đổi này giảm chi phí tính toán mà không làm giảm hiệu suất, khiến Xception trở thành lựa chọn phổ biến trong các ứng dụng thị giác máy tính hiện đại [6].

Xception được chia thành 3 phần chính: luồng vào, luồng giữa và luồng ra với các kết nối bỏ qua xung quanh 36 lớp.



Ảnh 2.2. Kiến trúc của Xception

2.3.1. Luồng vào (Entry flow):

Ảnh đầu vào với 3 kênh màu thông qua một lớp tích chập 3x3 được sử dụng với 32 bộ lọc làm giảm kích thước ảnh bản đầu và tăng số đặc trưng giúp mô hình nắm bắt được các đặc trưng cơ bản. Sau đó dùng hàm ReLU để tạo tính phi tuyến.

Tiếp theo, dữ liệu đầu vào tiếp tục đi qua 1 lớp tích chập 3x3 được sử dụng với 64 bộ lọc và hàm ReLU.

Sau khi trích xuất các đặc trưng cấp thấp ban đầu, lớp tích chập phân tách theo chiều sâu đã được áp dụng, cùng với lớp tích chập 1x1. Phương pháp maxpolling làm giảm kích thước bản đồ đặc trưng (feature map).

2.3.2. Luồng giữa (Middle flow)

Luồng giữa là phần quan trọng nhất của Xception, có nhiệm vụ học và củng cố các đặc trưng trừu tượng, cấp cao từ hình ảnh.

Luồng này gồm 3 lớp Depthwise Separable Convolution liên tiếp, mỗi lớp sử dụng 728 bộ lọc (filters) và kernel 3x3, kèm theo hàm kích hoạt ReLU sau mỗi lớp.

Điểm đặc biệt là mỗi khối này đều áp dụng kết nối tắt (residual connection). Đầu vào của khối được cộng trực tiếp với đầu ra của lớp tích chập thứ ba, tạo thành một khối dư (residual block). Cơ chế này giúp gradient truyền ngược hiệu quả hơn qua nhiều lớp, giảm thiểu vấn đề tiêu biến gradient (vanishing gradient) và cho phép mạng học sâu tới 36 lớp một cách ổn định

Khối kiến trúc trên được lặp lại 8 lần. Việc lặp lại này cho phép mạng phân tích sâu sắc các đặc trưng, từ đó nắm bắt được các mẫu hình phức tạp và bất biến (invariant

features) trong ảnh, là yếu tố then chốt để đạt độ chính xác cao trong các tác vụ thị giác máy tính.

2.3.3. *Luồng thoát (Exit flow)*

Luồng thoát có nhiệm vụ chuẩn bị các đặc trưng đã được học từ luồng giữa cho tác vụ phân loại cuối cùng.

Luồng này bao gồm một chuỗi các lớp Depthwise Separable Convolution với số lượng bộ lọc tăng dần ($728 \rightarrow 1024 \rightarrow 1536 \rightarrow 2048$), tất cả đều sử dụng kernel 3×3 . Sự gia tăng số lượng bộ lọc này cho phép mạng học các biểu diễn đặc trưng ngày càng phong phú và đa dạng hơn ở các lớp cuối. Sau đó sử dụng Global Average Pooling để tóm tắt toàn bộ bản đồ đặc trưng thành một vector duy nhất.

Thay vì sử dụng các lớp Fully Connected dày đặc tham số, Xception áp dụng Global Average Pooling. Phép toán này lấy giá trị trung bình của toàn bộ mỗi bản đồ đặc trưng 2D (feature map) và nén nó thành một giá trị vô hướng đơn lẻ. Kết quả là một vector đặc trưng 1D với chiều dài bằng số lượng bộ lọc của lớp cuối cùng (2048). Phương pháp này không chỉ giảm mạnh số lượng tham số, chống overfitting, mà còn buộc mạng phải học các đặc trưng mang tính toàn cục hơn.

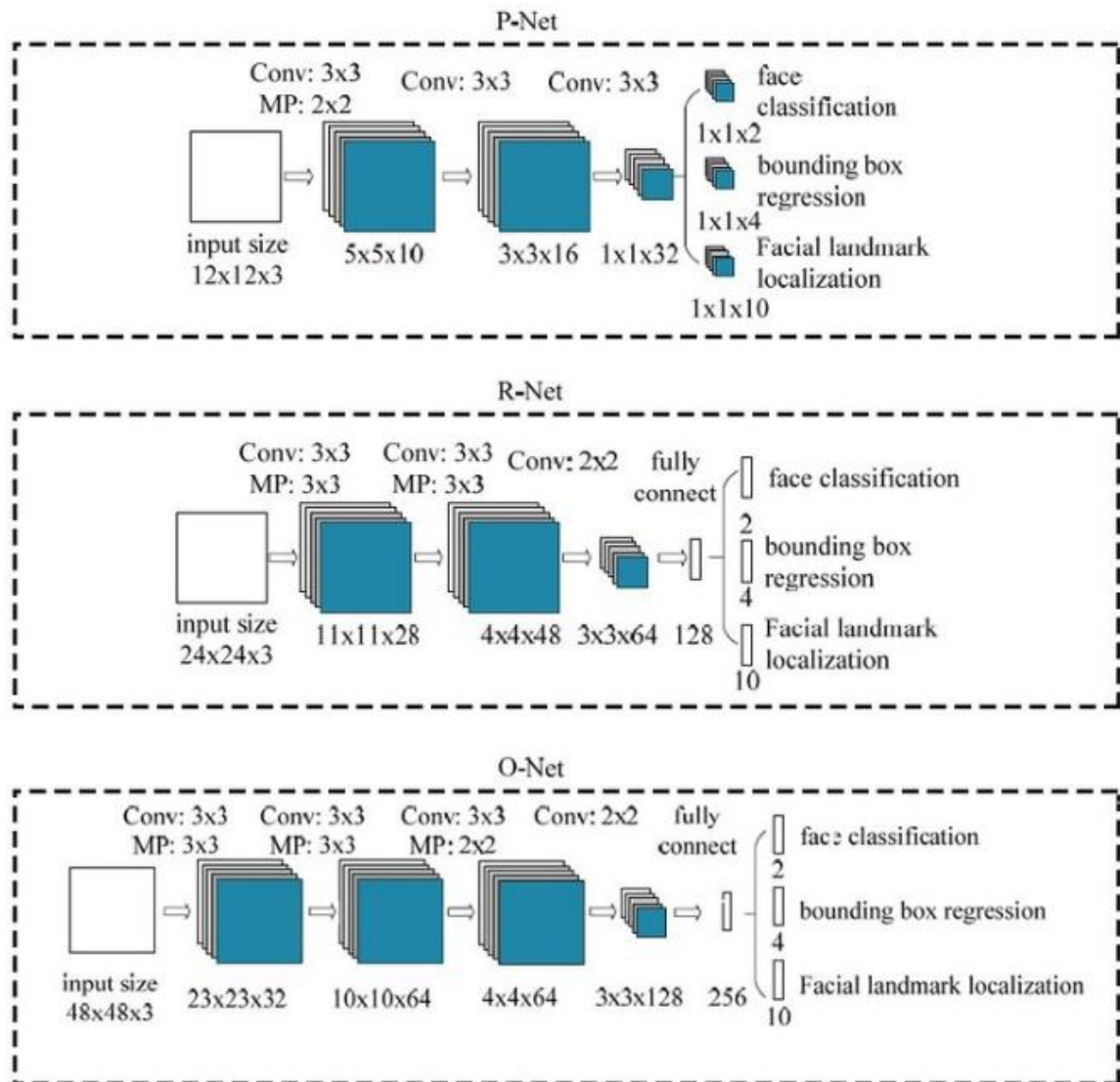
Vector đặc trưng 2048 chiều này sau đó được đưa vào một lớp Fully Connected duy nhất. Và tới lớp cuối cùng dùng để phân loại các nhãn phụ thuộc theo bài toán.

2.4. MTCNN

MTCNN được giới thiệu lần đầu trong một bài báo vào năm 2016 có tiêu đề “*Joint Face Detection and Alignment Using Multitask Cascaded Convolutional Networks*” bởi Kaipeng Zhang, Zhanpeng Zhang, Zhifeng Li, và Yu Qiao. Công trình được xuất bản trên IEEE Signal Processing Letters và sau đó được trình bày tại Hội nghị Quốc tế về Thị giác Máy tính IEEE 2017 (ICCV) [10].

Mô hình này được phổ biến nhanh chóng nhờ khả năng thực hiện cả việc phát hiện khuôn mặt và căn chỉnh các điểm mốc trên khuôn mặt trong một quy trình duy nhất. Nó được thiết kế để phát hiện khuôn mặt hiệu quả ở các tỷ lệ và hướng khác nhau đồng thời dự đoán các điểm mốc chính trên khuôn mặt như mắt, mũi và miệng. Cách tiếp cận đa nhiệm này đã giảm chi phí tính toán so với việc chạy các mô hình riêng biệt cho việc phát hiện và căn chỉnh khuôn mặt.

MTCNN sử dụng 3 mạng nơ-ron tích chập (CNN) hoạt động nối tiếp nhau để liên tục cải thiện các đề xuất khuôn mặt và phát hiện các điểm mốc chính. Ba mạng này gồm: P-Net (Proposal Network), R-Net (Refine Network) và O-Net (Output Network).



Ảnh 2.3. Kiến trúc của MTCNN

2.4.1. P-Net (Proposal Network):

Mạng này quét hình ảnh ở nhiều tỷ lệ khác nhau để tạo ra các vùng khuôn mặt tiềm năng (bounding boxes).

P-Net có kiến trúc nông với 3 lớp Conv 3×3. Lớp tích chập đầu tiên trích xuất các đặc trưng cơ bản nhất từ ảnh đầu vào như cạnh, góc, và texture đơn giản cho phép quét nhanh toàn bộ ảnh ở nhiều tỷ lệ khác nhau, sau đó dữ liệu đi qua 1 lớp PreLU và 1 lớp MaxPooling để thêm tính phi tuyến và giảm kích thước dữ liệu.

Lớp Conv thứ hai tiếp tục trích xuất các đặc trưng trừu tượng hơn từ đầu ra của lớp trước, và đi qua lớp PreLU để thêm tính phi tuyến.

Lớp tích chập cuối cùng trong phần trích xuất đặc trưng, tạo ra các đặc trưng đủ mạnh để thực hiện ba tác vụ và đi qua lớp PreLU để chuẩn bị đặc trưng cho các đầu ra

đa nhiệm. Cuối cùng, đầu ra lớp Conv 1×1 thực hiện đa nhiệm: phân loại face/non-face, hồi quy bounding box, và dự đoán landmark.

2.4.2. R-Net (Refine Network):

Sau đó R-Net nhận các vùng khuôn mặt tiềm năng từ P-Net (đã được crop và resize về 24×24 pixel), tinh chỉnh chúng bằng cách lọc các kết quả dương tính giả (false positive) và lọc ra các bounding boxes chất lượng cao hơn.

R-Net có kiến trúc sâu hơn với 3 lớp Conv chính. Lớp Conv đầu tiên với 28 filters trích xuất đặc trưng chi tiết hơn từ vùng ảnh đã được cắt, sau đó đi qua PReLU và MaxPooling với kernel 3×3 để thêm tính phi tuyến và tăng vùng tiếp nhận.

Lớp Conv thứ hai với 48 filters học các đặc trưng cấp cao hơn, tiếp tục qua PReLU và MaxPooling.

Lớp Conv cuối cùng này tiếp tục trích xuất đặc trưng với kernel 2×2 và 64 filters phù hợp với kích thước feature map nhỏ còn lại, tránh việc kernel quá lớn so với feature map nhỏ, đảm bảo phép tích chập vẫn thu thập thông tin hiệu quả mà không bị mất quá nhiều chi tiết rồi tiếp tục qua PReLU.

Phần quan trọng là Fully Connected layer với 128 units chuyển đặc trưng 2D thành vector 1D, tổng hợp thông tin toàn cục từ toàn bộ vùng ảnh. Cuối cùng, ba đầu ra FC thực hiện phân loại, hồi quy bounding box và dự đoán landmark dựa trên biểu diễn toàn cục nên chính xác hơn P-Net.

2.4.3. O-Net (Output Network):

Giai đoạn cuối cùng, O-Net thực hiện tinh chỉnh các hộp giới hạn (bounding boxes) và xác nhận khuôn mặt cuối cùng, đồng thời xác định chính xác năm điểm mốc trên khuôn mặt bao gồm: hai mắt, mũi và hai góc miệng.

O-Net là mạng có kiến trúc sâu và phức tạp nhất trong bộ ba mạng, với đầu vào được chuẩn hóa về kích thước 48×48 pixel. Mạng này bao gồm 4 lớp tích chập (Conv) chính.

Lớp Conv đầu tiên với 32 filters khởi động quá trình trích xuất đặc trưng từ ảnh có độ phân giải cao, sau đó dữ liệu được đưa qua hàm kích hoạt PReLU và lớp MaxPooling.

Lớp Conv thứ hai với 64 filters tiếp tục học các đặc trưng phức tạp hơn, rồi cũng đi qua PReLU và MaxPooling.

Lớp Conv thứ ba tiếp tục củng cố các đặc trưng này với 64 filters, rồi qua PReLU và MaxPooling sử dụng kernel 2×2 .

Lớp Conv cuối cùng với kernel 2×2 và 128 filters có nhiệm vụ trích xuất các đặc trưng cấp cao nhất, trước khi qua PReLU.

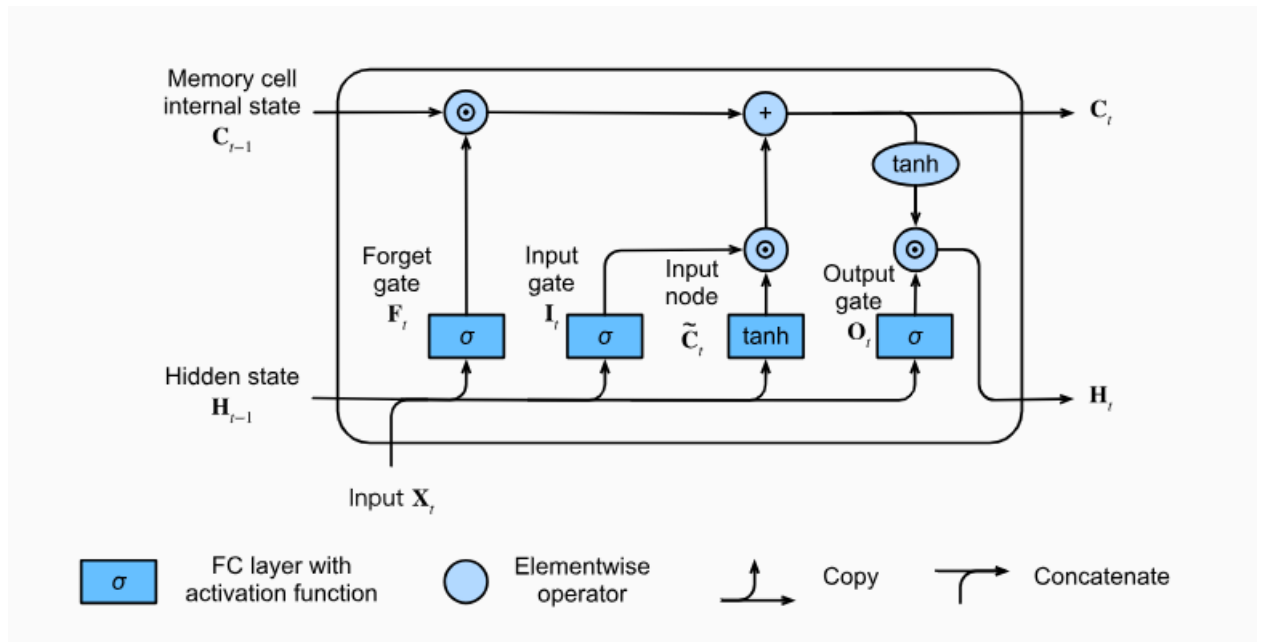
Dữ liệu tiếp tục tới Fully Connected layer với 256 units, có nhiệm vụ chuyển đổi đặc trưng 2D thành vector đặc trưng 1D với 256 chiều, tạo ra một biểu diễn toàn cục giàu thông tin.

Ba đầu ra từ các lớp Fully Connected cuối cùng là định vị điểm mốc (landmark localization) với 10 units để dự đoán tọa độ của 5 điểm mốc, cùng với các đầu ra cho phân loại khuôn mặt và hồi quy bounding box được tinh chỉnh với độ chính xác cao nhất.

Toàn bộ quy trình mang tính phân cấp, với mỗi mạng tập trung vào các nhiệm vụ chính xác hơn khi đề xuất tiền gần đến việc phát hiện khuôn mặt cuối cùng. Cấu trúc xếp tầng này giúp cân bằng giữa độ chính xác và tốc độ, đảm bảo hiệu suất cao ngay cả trong các điều kiện thử thách như ánh sáng thay đổi, tư thế và che khuất khuôn mặt.

2.5. Long Short-Term Memory (LSTM)

LSTM được đề xuất bởi Hochreiter & Schmidhuber vào năm 1997 [11], nó là một mạng RNN được thiết kế để học hỏi dữ liệu một cách tuần tự đồng thời khắc phục các hạn chế của mạng RNN truyền thống, đó là quên các thông tin quan trọng ở xa. Với các cell state (trạng thái ô), nó đóng vai trò như 1 đường truyền tín hiệu thông suốt giúp mô hình lưu giữ được thông tin dài hạn, làm cho chúng có hiệu quả cao với các nhiệm vụ liên quan tới các mẫu dữ liệu theo thời gian dài.



Ảnh 2.4. Kiến trúc của LSTM

Mạng LSTM bao gồm bốn thành phần cốt lõi, được gọi là các cổng (gates), có nhiệm vụ điều tiết và kiểm soát luồng thông tin trong quá trình lan truyền qua mạng. Mỗi

cổng sử dụng hàm kích hoạt Sigmoid để tạo ra các hệ số có giá trị nằm trong khoảng $[0,1]$, qua đó quyết định mức độ thông tin được duy trì hoặc loại bỏ. Khi giá trị đầu ra của cổng tiến gần đến 1, thông tin tương ứng sẽ được giữ lại và truyền tiếp; ngược lại, các giá trị gần 0 sẽ hạn chế hoặc chặn hoàn toàn thông tin không quan trọng.

2.5.1. Cổng quên (Forget Gate):

Cổng quên đảm nhiệm vai trò quyết định thông tin nào cần được giữ lại từ trạng thái ô trước đó. Cổng này nhận đầu vào là trạng thái ẩn trước h_{t-1} và đầu vào hiện tại x_t . Hai vector này được ghép nối (concatenate) thành một vector duy nhất, sau đó trải qua một phép biến đổi tuyến tính kết hợp với hàm kích hoạt Sigmoid:

$$f_t = \sigma(W_f \times [h_{t-1}, x_t] + b_f) \quad (6)$$

Trong đó:

- σ là hàm Sigmoid
- W_f là ma trận trọng số
- $[h_{t-1}, x_t]$ biểu thị phép ghép nối hai vector thành một vector
- b_f là độ lệch (bias)

Giá trị f_t thu được là một vector với các phần tử trong khoảng $[0, 1]$, đóng vai trò như một bộ lọc thông minh. Khi giá trị f_t gần 1 giữ lại thông tin, ngược lại giá trị gần 0 loại bỏ thông tin. Đây là cơ chế chọn lọc thông minh giúp LSTM quên đi dữ liệu không cần thiết, tạo không gian cho việc học các thông tin mới.

2.5.2. Cổng vào (Input Gate):

Cổng đầu vào sẽ thực hiện hai chức năng song song. Đầu tiên, nó xác định mức độ quan trọng của thông tin mới bằng cách sử dụng hàm Sigmoid, tương tự như cổng quên nhưng với bộ trọng số riêng biệt

$$i_t = \sigma(W_i \times [h_{t-1}, x_t] + b_i) \quad (7)$$

Giá trị $i_t \in [0,1]$, giá trị càng gần 1 cho thấy thông tin mới càng quan trọng và cần được ghi nhiều vào bộ nhớ. Sau đó nó sử dụng hàm tanh để tạo ra giá trị ứng viên (candidate value) cho cell state mới:

$$\tilde{C}_t = \tanh(W_c \times [h_{t-1}, x_t] + b_c) \quad (8)$$

Hàm tanh cho đầu ra trong khoảng $[-1, 1]$, cho phép biểu diễn cả thông tin âm và dương một cách cân bằng. Vector $\tilde{C}_t$ chứa các giá trị tiềm năng có thể được thêm vào cell state, đại diện cho nội dung thông tin mới thuần túy chưa qua đánh giá mức độ quan trọng.

Sự kết hợp giữa hai thành phần này tạo nên một cơ chế linh hoạt: i_t quyết định "có nên ghi bao nhiêu trong khi $\tilde{C}_t$ xác định "nên ghi cái gì". Phép nhân từng phần tử giữa i_t và $\tilde{C}_t$ sẽ tạo ra thông tin mới đã được điều chỉnh theo mức độ quan trọng trước khi thêm vào cell state

2.5.3. Cập nhật cell state (Memory Highway)

Quá trình cập nhật cell state là trung tâm của cơ chế bộ nhớ dài hạn trong LSTM, thực hiện sự kết hợp có trọng số giữa thông tin cũ và thông tin mới.

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (9)$$

Trong đó:

- $\odot$ là phép nhân từng phần tử

Phương trình này bao gồm hai thành phần chính: phần thứ nhất $f_t \odot C_{t-1}$ thực hiện việc lọc thông tin cũ dựa trên cổng quên. Mỗi phần tử của cell state cũ C_{t-1} được nhân với giá trị tương ứng trong f_t , kết quả là những thông tin quan trọng được giữ lại trong khi những thông tin không cần thiết bị loại bỏ. Phần thứ hai $i_t \odot \tilde{C}_t$ đại diện cho thông tin mới đã được điều chỉnh mức độ quan trọng. Mỗi phần tử của giá trị ứng viên $\tilde{C}_t$ được nhân với giá trị tương ứng trong i_t , đảm bảo chỉ những thông tin quan trọng mới được thêm vào với trọng số thích hợp.

Nhờ đó, gradient có thể truyền trực tiếp qua phép cộng trong phương trình (9). Cơ chế này bảo toàn thông tin gradient qua hàng trăm thậm chí hàng ngàn bước thời gian, cho phép LSTM học được các phụ thuộc dài hạn trong dữ liệu chuỗi.

2.5.4. Cổng đầu ra (Output Gate)

Cổng đầu ra đảm nhiệm vai trò quyết định phần nào của cell state sẽ được xuất ra làm hidden state hiện tại, đồng thời chuẩn bị thông tin cho các bước xử lý tiếp theo. Cổng này sử dụng hàm Sigmoid để tạo ra một bộ lọc thông minh dựa trên trạng thái ẩn trước đó và đầu vào hiện tại.

$$o_t = \sigma(W_o \times [h_{t-1}, x_t] + b_o) \quad (10)$$

Sau khi có được bộ lọc o_t , LSTM áp dụng hàm tanh lên cell state hiện tại để tạo ra hidden state mới:

$$h_t = o_t \odot \tanh(C_t) \quad (11)$$

Hàm tanh trong phương trình này đảm bảo các giá trị trong cell state được chuẩn hóa về một phạm vi cố định trước khi xuất ra, trong khi phép nhân với o_t thực hiện việc lọc chọn lọc: chỉ những phần tử có giá trị o_t gần 1 mới được giữ lại đáng kể trong hidden state

Hidden state h_t thu được đóng vai trò vừa là đầu ra hiện tại, vừa được truyền sang bước thời gian tiếp theo.

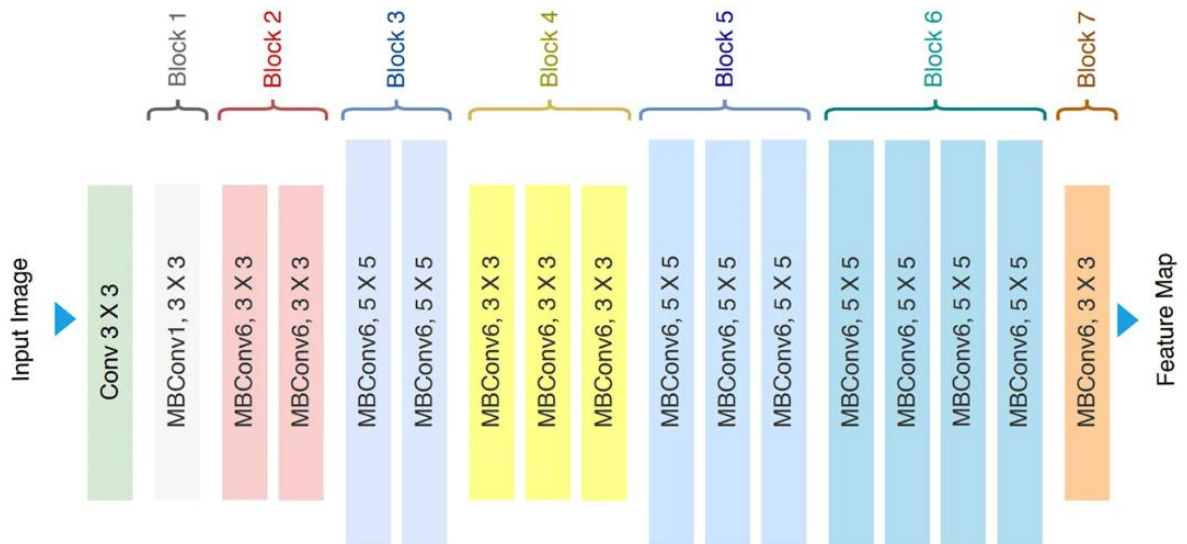
Trích dẫn: Các công thức (6), (7), (8), (9), (10), (11) được trình bày trong GeeksforGeeks, "*What is LSTM - Long Short Term Memory?*" [11].

2.6. EfficientNet

EfficientNet là một mạng nơ-ron tích chập (CNN) nhằm đạt được hiệu suất cao với ít tài nguyên tính toán hơn so với các kiến trúc trước đó. Nó được giới thiệu bởi Mingxing Tan và Quoc V. Le từ Google Research trong bài báo năm 2019 của họ "*EfficientNet: Suy nghĩ lại về quy mô mô hình cho mạng nơ-ron tích chập*" [12]. Ý tưởng cốt lõi đằng sau EfficientNet là phương pháp Compound Scaling [12], nó giúp mô hình học có hệ thống và cân bằng theo 3 chiều: chiều sâu, chiều rộng, và độ phân giải ảnh đầu vào.

Trong đó, kiến trúc nền tảng của mạng EfficientNet là EfficientNetB0 [13], nó được tác giả thiết kế là một mô hình cực kì tối ưu bằng phương pháp Neural Architecture Search (NAS) [14]:

- Thành phần cốt lõi của mạng EfficientNetB0 được cấu tạo từ nhiều MBConv
- Bên trong mỗi khối MBConv, mô hình sử dụng squeeze-and-excitation (SE Block) để giúp mô hình tự động học các điểm quan trọng của ảnh.
- Mô hình EfficientNetB0 sử dụng hàm kích hoạt Swish, hàm này sử dụng Sigmoid thay vì sử dụng ReLU truyền thống giúp nó phát hiện các mẫu trong ảnh tốt hơn các phương pháp cũ.
- Ngoài ra nó còn sử dụng DropConnect để tắt ngẫu nhiên một số kết nối trong quá trình huấn luyện giúp giảm khả năng bị overfitting của mô hình.



Ảnh 2.5. Kiến trúc mô hình EfficientNetB0

Hình 2.5 minh họa kiến trúc tổng thể của mô hình EfficientNet-B0. Kiến trúc này được xây dựng dựa trên các khối MBConv (Mobile Inverted Bottleneck Convolution) kết hợp với cơ chế compound scaling.

Lớp đầu tiên của mô hình là một Conv 3×3 với stride 2×2 , lớp này có nhiệm vụ trích xuất những đặc trưng cơ bản và cấp thấp từ hình ảnh, như cạnh, góc và kết cấu, đồng thời giảm kích thước và tăng số kênh đặc trưng đầu vào ban đầu.

Sau lớp tích chập mở đầu, phần thân của EfficientNet-B0 được cấu thành từ 7 block chính (Block 1 đến Block 7), mỗi block gồm một hoặc nhiều MBConv với kích thước kernel và hệ số mở rộng khác nhau.

Block 1 đóng vai trò khởi đầu cho phần thân của mạng, sử dụng MBConv1 với kernel 3×3 . Khối này không thực hiện bước mở rộng kênh, mà tập trung trích xuất các đặc trưng mức thấp và chuyển tiếp mượt mà từ lớp tích chập ban đầu sang các tầng sâu hơn của mạng.

Tiếp theo là giai đoạn mở rộng (Expansion), giai đoạn này tăng số kênh đặc trưng lên một hệ số nhất định (ví dụ MBConv6 mở rộng gấp 6 lần) giúp tăng khả năng biểu diễn.

Sau bước mở rộng là giai đoạn lọc chiều sâu (Depthwise Filtering), giai đoạn này bao gồm các khối đều là MBConv6, 3×3 hoặc MBConv6, 5×5 . Đây là bộ lọc chính, hoạt động độc lập trên từng kênh đặc trưng đã được mở rộng, giúp giảm đáng kể tham số so với tích chập thông thường.

Trong giai đoạn đầu (Block 2 đến 4), mạng áp dụng phương pháp trích xuất đặc trưng đa tỷ lệ (multi-scale feature extraction) bằng cách luân phiên sử dụng kernel 3×3 và

5x5. Việc này cho phép mạng đồng thời nắm bắt cả các đặc trưng cục bộ chi tiết (với 3x3) và các cấu trúc cục bộ lớn hơn (với 5x5) ngay từ những tầng xử lý ban đầu, tạo ra một biểu diễn đặc trưng phong phú và vững chắc làm nền tảng.

Khi tiến vào các tầng sâu hơn (Block 5 đến 7), mạng chuyển hướng sang ưu tiên tích hợp ngữ cảnh toàn cục (global context integration) bằng cách chủ yếu sử dụng kernel 5x5. Ở giai đoạn này, đặc trưng đã trở nên trừu tượng và kích thước không gian đã bị thu nhỏ, do đó một kernel lớn hơn có khả năng bao quát một vùng rộng trên ảnh gốc. Điều này là cần thiết để mô hình nắm bắt các mối quan hệ phụ thuộc dài hạn (long-range dependencies), hiểu được cấu trúc tổng thể và bối cảnh phức tạp của toàn bộ cảnh, từ đó đưa ra các dự đoán cấp cao chính xác hơn. Sự chuyển đổi này phản ánh nguyên lý xử lý thị giác phân cấp: từ chi tiết cục bộ ở tầng thấp đến ngữ nghĩa toàn cục ở tầng sâu.

Và giai đoạn cuối cùng là giai đoạn nén (Squeeze/Projection), giai đoạn này sử dụng tích chập 1x1 khác để giảm số kênh đặc trưng trở lại, tạo ra "cổ chai" (bottleneck). Giai đoạn này thường kết hợp với kỹ thuật SELayer (Squeeze-and-Excitation) để học trọng số cho từng kênh, tăng cường khả năng biểu diễn.

EfficientNet-B0 có cấu trúc tối ưu và phân cấp rõ ràng. Mô hình sử dụng các khối MBConv theo nguyên lý "cổ chai đảo ngược", kết hợp lọc chiều sâu và cơ chế Squeeze-and-Excitation. Điểm thông minh nằm ở chiến lược sử dụng kernel: các tầng đầu dùng đa tỷ lệ (3x3 và 5x5) để bắt đặc trưng cục bộ, còn tầng sâu ưu tiên kernel 5x5 để tích hợp ngữ cảnh toàn cục. Nhờ cấu trúc này giúp mạng vừa 'nhìn' rõ các chi tiết nhỏ, vừa hiểu được bức tranh toàn cảnh mà không làm bộ máy tính toán trở nên nặng nề.

CHƯƠNG 3. TIỀN XỬ LÝ DỮ LIỆU

3.1. Tổng quan về dữ liệu

Dữ liệu được sử dụng trong chủ đề này là FaceForensics++, bộ dữ liệu này được 1 nhóm nghiên cứu tại Technical University of Munich (TUM) công bố vào năm 2019. FaceForensics++ bao gồm 1000 video gốc lấy từ YouTube, mỗi video dài khoảng 10 đến 20 giây [4]. Từ 1000 video gốc (original) này sử dụng 1 vài phương pháp để tạo ra các kỹ thuật deepfake khác bao gồm :

- Deepfakes (1000 deepfake videos): Hoán đổi khuôn mặt bằng autoencoder..
- Face2Face 1000 deepfake videos: Thay đổi biểu cảm khuôn mặt.
- FaceShifter (1000 deepfake videos): Hoán đổi khuôn mặt chất lượng cao sử dụng mạng GAN.
- FaceSwap (1000 deepfake videos): Thay khuôn mặt dựa trên landmark.
- NeuralTextures (1000 deepfake videos): Tạo kết cấu (texture) động cho khuôn mặt bằng mạng neural.
- DeepFakeDetection (1000 deepfake videos): Kỹ thuật này sử dụng các mô hình deep learning chủ yếu là mạng GAN hoặc autoencoder, các video này sẽ bao gồm nhiều người cùng 1 video khác với 5 kỹ thuật deepfake trên chỉ chứa 1 khuôn mặt rõ ràng.



Ảnh 3.1. Bộ dữ liệu FaceForensics++

Hình ảnh minh họa thể hiện sự biến đổi của cùng một đối tượng từ hình ảnh gốc sang các biến thể giả mạo khác nhau có trong bộ dữ liệu:

- Pristine: Đây là hình ảnh gốc, chưa qua chỉnh sửa, dùng làm dữ liệu đối chứng.
- DeepFakes: Thay thế hoàn toàn khuôn mặt bằng AI.
- Face2Face: Áp đặt cử động của người này lên người kia.
- FaceSwap: Thao túng hoán đổi khuôn mặt truyền thống.
- NeuralTextures: Chỉnh sửa chi tiết bề mặt và vùng miệng để khớp với lời thoại.

Ngoài 7000 video, dữ liệu còn thêm 10 file csv, các file này mỗi file lưu trữ đường dẫn, nhãn, số frame, chiều dài, chiều rộng, dung lượng,... của các video với các kỹ thuật deepfake. Tổng dữ liệu chứa 7010 files với 7000 videos (6000 deepfake, 1000 real) và 10 file csv.

3.2. Tiền xử lý dữ liệu

Để mô hình có thể học tốt và đạt hiệu quả cao hơn, vì vậy cần thực hiện các bước tiền xử lý dữ liệu trước khi đưa vào huấn luyện như sau:

3.2.1. Chia dữ liệu

Dữ liệu sẽ được chia thành các tập Train, Test, Validation phục vụ cho việc huấn luyện, kiểm định và đánh giá mô hình:

- Tập train chiếm 70% dữ liệu.
- Tập Test chiếm 15% dữ liệu.
- Tập Validation chiếm 15% dữ liệu.

Với 70% dữ liệu được sử dụng cho tập huấn luyện (khoảng 42.000 ảnh), lượng dữ liệu này đủ lớn để mô hình học được các đặc trưng quan trọng, đồng thời giúp giảm nguy cơ overfitting. Và với tập test 15% (khoảng 9.000 ảnh) giúp đánh giá hiệu quả hơn khả năng nhận diện trên các dữ liệu chưa từng xuất hiện trong quá trình huấn luyện của mô hình.

3.2.2. Chia lại nhãn

Dữ liệu ban đầu bao gồm 7 thư mục, mỗi thư mục đại diện cho 1 nhãn, mỗi nhãn này chứa 1000 video, do phạm vi của nguyên cứu chỉ tập trung vào bài toán nhị phân (phân biệt 2 nhãn real hoặc fake), vì vậy nhóm chia lại cấu trúc thư mục như sau để tiện huấn luyện:

- Thư mục original chứa 1000 video gốc sẽ được gán nhãn real.
- Thư mục DeepFakeDetection sẽ không sử dụng do dữ liệu này dùng các phương pháp trùng với các phương pháp khác nên không dùng nhãn này.

- Các thư mục còn lại sẽ gộp chung vào thành nhân fake với 5000 video.

Mặc dù việc gộp này có thể dẫn đến hiện tượng mô hình thiên lệch về lớp chiếm ưu thế, khiến mô hình đạt độ chính xác tổng thể cao nhưng giảm khả năng nhận diện chính xác lớp thiểu số, từ đó làm tăng nguy cơ misclassification đối với các video real, nhưng lại giúp giảm độ khó của bài toán cũng như phù hợp với mục tiêu chính của đề tài lại phát hiện video thật hay sử dụng video giả mạo.

3.2.3. Trích xuất khung hình

Tiếp theo các video này sẽ được trích xuất 10 khung hình chia đều từ đầu đến cuối video đại diện cho mỗi 1 video, cách làm này giúp giảm tính toán đáng kể cho mô hình, đảm bảo tính đại diện cho mỗi video, và tăng thêm tính đa dạng về dữ liệu cho mô hình.

Hơn nữa 10 khung hình này có thể cũng cấp đủ ngữ cảnh và đặc trưng cần thiết cho mô hình tính toán phân loại real hay fake dựa theo tính tiếp nối và sự nhất quán giữa các khung hình của các khung hình trong 1 video, nó giúp mô hình nhận diện hiệu quả các bất thường về không gian và thời gian trong các video deepfake, từ đó nâng cao chất lượng của mô hình.

Để dễ dàng quản lý, các video này sẽ đại diện cho một thư mục riêng, mỗi thư mục sẽ chứa 10 khung hình sau trích xuất tương ứng với video tương ứng.

3.2.4. Phát hiện và cắt khuôn mặt

Những khung hình sau khi trích xuất này không chỉ chứa khuôn mặt người mà còn cả không gian xung quanh người đó nữa vì vậy sử dụng mạng MTCNN để xác định và cắt khuôn mặt cho từng người, các ảnh sau khi cắt thống nhất về kích thước 224x224 pixels.

3.2.5. Căn chỉnh khuôn mặt

Bên cạnh cắt khuôn mặt, dựa trên 5 điểm mốc mà MTCNN cung cấp bao gồm: mắt trái, mắt phải, mũi, khóe miệng trái, khóe miệng phải. Các khuôn mặt sau khi cắt sẽ được căn chỉnh và xoay khuôn mặt cùng một tư thế thẳng.



Ảnh 3.2. Phát hiện 5 điểm landmark trên khuôn mặt bằng mô hình MTCNN

Việc này giúp toàn bộ dữ liệu được chuẩn hóa 1 về kiểu giúp mô hình có thể được được các thay đổi của khuôn mặt qua các khung hình 1 các dễ dàng hơn, làm tăng hiệu suất cho mô hình.

3.2.6. Tăng cường dữ liệu

Để tăng tính đa dạng cho dữ liệu và giảm hiện tượng quá khớp (overfitting), vì vậy nhóm đã áp dụng các biện pháp để tăng cường dữ liệu bao gồm:

Phép lật ảnh theo phương ngang (Horizontal Flip), phương pháp này là một kỹ thuật quan trọng để mô phỏng việc đối tượng soi gương hoặc thay đổi góc nhìn từ trái sang phải, Thông thường phép lật này được áp dụng với xác suất $p = 0.5$ (50%) để đảm bảo mô hình học được cả hai hướng nhìn một cách cân bằng. Điều này giúp mô hình không bị phụ thuộc vào hướng của khuôn mặt:

$$I'(x, y) = I(W - x - 1, y) \quad (12)$$

Trong đó:

- W là chiều rộng của ảnh.
- I và I' lần lượt là ảnh gốc và ảnh sau biến đổi.
- x và y lần lượt là tọa độ theo chiều ngang và chiều dọc của một điểm ảnh (pixel)

Điều chỉnh ngẫu nhiên độ sáng (brightness), Việc thay đổi độ sáng giúp mô hình hoạt động ổn định trong các điều kiện ánh sáng khác nhau (ngoài trời nắng gắt, trong phòng tối, hoặc dưới ánh đèn phức tạp.

$$I'(x, y) = I(x, y) + \beta \quad (13)$$

Trong đó:

- Hệ số β : hệ số bù độ sáng

Điều chỉnh ngẫu nhiên độ tương phản (contrast), nó ảnh hưởng đến khả năng phân biệt giữa các chi tiết trên khuôn mặt và nền ảnh. Việc điều chỉnh độ tương phản giúp mô hình thích nghi với các mức độ rõ nét khác nhau của ảnh đầu vào.

$$I'(x, y) = \alpha \cdot I(x, y) \quad (14)$$

Trong đó:

- Hệ số α : hệ số tỷ lệ độ tương phản.

Điều chỉnh ngẫu nhiên độ đậm màu. (saturation), nó quyết định mức độ rực rỡ của màu sắc. Việc điều chỉnh này giúp mô hình nhận diện tốt ngay cả khi ảnh bị bạc màu hoặc có màu sắc quá chói.

$$I'(x, y) = \gamma \cdot I(x, y) + (1 - \gamma) \cdot I_{\text{gray}}(x, y) \quad (15)$$

Trong đó:

- I_{gray} là ảnh chuyển sang không gian xám
- γ là hệ số điều chỉnh độ đậm màu

Việc tăng cường này giúp giả lập các trường hợp có thể gặp trong thực tế ví dụ như: sự thay đổi của môi trường, góc chụp, ánh sáng,... làm thay đổi chất lượng ảnh/video.

Trích dẫn: Các công thức (12), (13), (14), (15) được trích xuất từ bài *Digital Image Processing*, 4th Edition, Pearson, 2018 của R. C. Gonzalez and R. E. Woods, [15].



Ảnh 3.3. Ví dụ ảnh đã được tăng cường

Hình minh họa cho thấy ba ảnh được tạo ra từ cùng một ảnh gốc thông qua các phép tăng cường dữ liệu khác nhau.

- Augmented 1: Ảnh được lật ngang, đồng thời giảm độ sáng và độ tương phản, độ đậm màu tăng nhẹ so với ảnh gốc.

- Augmented 2: Ảnh giữ nguyên hướng, giảm nhẹ độ sáng và tăng nhẹ độ đậm màu, trong khi độ tương phản gần như không thay đổi.
- Augmented 3: Ảnh giữ nguyên hướng, giảm độ sáng và độ tương phản, độ đậm màu gần như giữ nguyên.

Tăng cường dữ liệu đối với mỗi 10 khung hình thuộc 1 video sẽ có kiểu tăng cường như nhau để đảm bảo tính nhất quán theo thời gian, còn đối với mỗi video sẽ ngẫu nhiên dựa trên 4 yếu tố trên:

- Có lật ảnh theo phương ngang hay không.
- Độ sáng ngẫu nhiên trong khoảng $[-0.15, +0.15]$.
- Độ tương phản ngẫu nhiên trong khoảng $[-0.15, +0.15]$.
- Độ đậm màu ngẫu nhiên trong khoảng $[-0.1, +0.1]$.

CHƯƠNG 4. XÂY DỰNG VÀ HUẤN LUYỆN MÔ HÌNH

Mục tiêu của khóa luận tốt nghiệp là phát hiện deepfake của ảnh và video vì vậy mô hình không chỉ cần nắm bắt được không gian dựa theo ảnh từng frame mà còn phải nắm bắt được biến đổi theo thời gian giữa từng frame. Do đó ý tưởng chính của mô hình chính là kết hợp một mạng CNN để phân tích không gian trong từng frame với một phương pháp để nắm bắt mối quan hệ theo thời gian giữa các khung hình.

4.1. Xây dựng mô hình EfficientNet kết hợp Temporal Attention

Giải pháp ban đầu đề xuất đó là sử dụng mạng EfficientNet làm mạng CNN với nhiệm vụ trích xuất đặc trưng không gian cho từng khung hình, sau đó các đặc trưng sẽ thông qua Temporal Attention để học các trọng số tương ứng với mức độ quan trọng của mỗi khung hình từ đó thấy được tính không nhất quán của khuôn mặt theo thời gian.



Ảnh 4.1. Kiến trúc mô hình EfficientNet + Temporal Attention

Hình 4.1 mô tả kiến trúc của mô hình thông qua các mô-đun:

- Đầu vào (Video Frames): Video được biểu diễn thành chuỗi khung hình RGB với kích thước ($B \times T, 3, H, W$), trong đó B là batch, T là số khung hình, 3 là 3 kênh màu RGB, và (H, W) là kích thước không gian của ảnh.
- EfficientNet: Từng khung hình được xử lý độc lập bởi mạng EfficientNet để thu được các đặc trưng cấp cao. Và giữ nguyên thứ tự thời gian.
- Vector Feature ($B \times T, 1408$): Đầu ra là các vector đặc trưng ($B \times T, 1408$) biểu diễn thông tin không gian của mỗi frame.
- Reshape ($B, T, 1408$): Các đặc trưng được tái cấu trúc lại để phù hợp với đầu vào của mô-đun xử lý chuỗi thời gian.
- Temporal Attention: Mô-đun này học và gán trọng số cho từng khung hình, cho phép mô hình tập trung vào các frame quan trọng và giảm thiểu ảnh hưởng của frame nhiễu.
- Fully Connected + Sigmoid: Đặc trưng video sau khi được tổng hợp thông qua cơ chế Temporal Attention được đưa qua một lớp fully connected và hàm Sigmoid để dự đoán xác suất cho bài toán phân loại nhị phân.

4.1.1. Trích xuất đặc trưng không gian bằng EfficientNet

Đầu vào của mô hình sẽ là 1 chuỗi các khung hình RGB với cấu hình 5 chiều (B, T, C, H, W):

$$X \in R^{B \times T \times C \times H \times W}$$

Trong đó:

- B : batch size
- T : số khung hình trong mỗi video
- C = 3 : số kênh màu RGB
- H, W : chiều cao và chiều rộng ảnh

Để trích xuất đặc trưng từ chuỗi video, dữ liệu ban đầu được tái cấu trúc (reshape) thành (B x T, C, H, W) để thỏa mãn yêu cầu đầu vào của mô hình rồi đưa vào mô hình EfficientNet. Sau khi đi qua mô hình mỗi khung hình được biểu diễn là một vector đặc trưng:

$$f_t \in R^{1408}$$

Trong đó:

- f : là viết tắt của feature vector (vector đặc trưng)
- t : đại diện cho thời gian hoặc thứ tự.

Sau khi qua mô hình EfficientNet dữ liệu sẽ có dạng (B x T, 1408), để tiếp tục vào Temporal Attention dữ liệu được reshape lại thành (B, T, 1408) giúp mô hình nhận diện được số khung hình của video đó.

4.1.2. Temporal Attention (Cơ chế chú ý theo thời gian)

Sau khi có được chuỗi đặc trưng không gian, mô hình áp dụng cơ chế Temporal Attention để tự động học cách tập trung vào những khung hình quan trọng nhất.

Quá trình tính toán của cơ chế này bao gồm 3 bước. Đầu tiên, nó sẽ tính điểm số attention của khung hình thứ t:

$$e_t = W^T \times f_t + b \quad (16)$$

Trong đó:

- W: là các trọng số
- f_t : là đặc trưng khung hình tại thời điểm t

- b : là các hệ số bias.

Sau đó hàm softmax được áp dụng trên chiều thời gian (T) để tính toán trọng số α . Trọng số này đại diện cho mức độ đóng góp của từng khung hình vào quyết định cuối cùng. Những khung hình có sự thay đổi bất thường hoặc chứa nhiều dấu vết deepfake sẽ nhận được trọng số cao hơn:

$$\alpha_t = \frac{\exp(e_t)}{\sum_{k=1}^T \exp(e_k)} \quad (17)$$

Trong đó:

- α_t là trọng số attention của khung hình thứ t ,
- e_k là điểm số attention của khung hình thứ k
- e_t là điểm số attention của khung hình thứ t

Cuối cùng là tổng hợp đặc trưng, vector đặc trưng đại diện cho toàn bộ video được tính bằng tổng có trọng số của các đặc trưng khung hình:

$$V_{final} = \sum_{t=1}^T \alpha_t \times f_t \quad (18)$$

Trong đó:

- f_t : là đặc trưng khung hình tại thời điểm t
- α_t : là trọng số của khung hình tại thời điểm t

Cơ chế Temporal Attention cho phép mô hình nắm bắt sự không nhất quán theo thời gian giữa các khung hình khuôn mặt, từ đó nâng cao khả năng phát hiện các hành vi giả mạo trong video deepfake.

Trích dẫn: Các công thức (16), (17), (18) được trích xuất dựa trên bài viết "*STA-TSN: Spatial-Temporal Attention Temporal Segment Network for action recognition in video*," Wang *et al* [16].

4.1.3. Phân loại và đưa ra quyết định

Sau khi tính tổng có trọng số, mô hình không còn là một chuỗi hình ảnh mà là một "Vector tổng hợp bằng chứng". Việc phân biệt dựa trên:

- Bộ lọc thông minh (Lớp FC): Vector này được đưa qua một lớp tuyến tính. Lớp này đóng vai trò như một bộ lọc, nó đã được học rằng: "Nếu trong tổng có chứa các dấu hiệu A (nét đứt), B (màu da bất thường), thì đó là giả".
- Chuyển đổi thành xác suất: Kết quả cuối cùng là một con số. Con số này càng lớn thì độ tin cậy video là Fake càng cao. Nếu con số nhỏ hoặc âm, mô hình kết luận video là Real.

4.2. Xây dựng mô hình Xception kết hợp mạng LSTM

Nhằm nâng cao khả năng học của mô hình, nhóm đã đề ra một phương pháp khác đó là: Thay mạng EfficientNet thành một mạng sâu hơn là Xception và thay thế phương pháp Temporal Attention thành mạng LSTM. Việc thay thế này giúp mô hình trích xuất các đặc trưng mạnh mẽ hơn mà còn khiến mô hình có thể ghi nhớ được sự biến đổi của khuôn mặt trong từng khung hình theo thời gian.



Ảnh 4.2. Kiến trúc mô hình Xception + LSTM

Hình 4.2 mô tả kiến trúc của mô hình thông qua các Mô-đun:

- Đầu vào (Video Frames): Video được biểu diễn thành chuỗi khung hình RGB với kích thước ($B \times T, 3, H, W$), trong đó B là batch, T là số khung hình, 3 là 3 kênh màu RGB, và (H, W) là kích thước không gian của ảnh.
- Xception: Từng khung hình được xử lý độc lập bởi mạng Xception để thu được các đặc trưng cấp cao. Và giữ nguyên thứ tự thời gian.
- Vector Feature ($B \times T, 2048$): Đầu ra là các vector đặc trưng ($B \times T, 2048$) biểu diễn thông tin không gian của mỗi frame.
- Reshape ($B, T, 2048$): Các đặc trưng được tái cấu trúc lại để phù hợp với đầu vào của mô-đun xử lý chuỗi thời gian.
- Mạng LSTM (Long Short-Term Memory) xử lý tuần tự chuỗi đặc trưng theo thời gian. Với cơ chế cổng (gate), LSTM có khả năng học các phụ thuộc dài hạn, lựa chọn ghi nhớ thông tin quan trọng và loại bỏ thông tin không cần thiết qua từng bước thời gian, từ đó nắm bắt ngữ cảnh động và diễn biến của video.
- Fully Connected + Sigmoid: Hidden state tại thời điểm cuối cùng của hai hướng (thuận và nghịch) trong LSTM được kết hợp lại và sử dụng làm đầu vào cho lớp

fully connected, tiếp theo là hàm kích hoạt Sigmoid nhằm ước lượng xác suất cho bài toán phân loại nhị phân

4.2.1. Trích xuất đặc trưng không gian bằng Xception

Tương tự với mạng EfficientNet, đầu vào của mô hình cũng là 1 chuỗi các khung hình RGB với cấu hình 5 chiều (B, T, C, H, W). Dữ liệu này cũng sẽ được tái cấu trúc (reshape) thành (B x T, C, H, W) để thỏa mãn yêu cầu đầu vào của mô hình rồi đưa vào mô hình.

Sau khi đi qua mô hình mỗi khung hình được biểu diễn là một vector đặc trưng ($f_t \in R^{2048}$) và sau đó đầu ra của mạng cũng sẽ được reshape từ (B x T, 2048) thành (B, T, 2048).

4.2.2. Phân tích chuỗi thời gian bằng LSTM

Khác với Temporal Attention chỉ phân biệt thông qua sự thay đổi lớn của một vài khung hình, LSTM ghi nhớ và học sự thay đổi của khuôn mặt trong từng khuôn hình theo thứ tự thời gian.

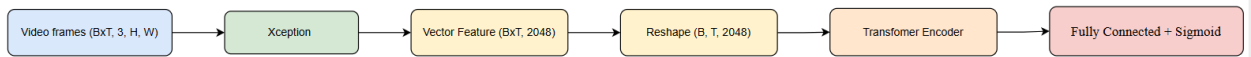
- LSTM sử dụng một lớp Bidirectional LSTM (LSTM hai chiều). Việc này giúp mô hình hiểu được ngữ cảnh của khung hình dựa trên cả khung hình trước và sau nó.
- Mô hình sẽ liên tục cập nhật trạng thái ẩn và trạng thái bộ nhớ, cho phép nó phát hiện các dấu hiệu và các chuyển động bất thường trên khuôn mặt.
- Cuối cùng mô hình sẽ lấy đầu ra là khung hình cuối cùng để đại diện cho toàn bộ đặc trưng của video. Vector này mang đầy đủ thông tin bất thường của toàn bộ video deepfake.

4.2.3. Phân loại và đưa ra quyết định

Sau khi tổng hợp thành một vector đại diện, dữ liệu sẽ tiếp tục đi qua một lớp Fully Connected, lớp này ánh xạ đặc trưng thành một giá trị logit duy nhất. Sau đó, giá trị logit này sẽ đi qua hàm Sigmoid để chuyển thành xác suất sử dụng deepfake của video.

4.3. Xây dựng mô hình Xception kết hợp Transformer Encoder

Tiếp nối các ý tưởng từ mô hình Xception kết hợp với LSTM, một kiến trúc khác đã được đưa ra với Xception vẫn được sử dụng nhưng thay vì sử dụng LSTM, mô hình sử dụng Transformer Encoder làm mô hình thay thế. Với sự thay thế này, mô hình sẽ học được mối quan hệ toàn cục giữa các khung hình thay vì chỉ xử lý tuần tự.



Ảnh 4.3. Kiến trúc mô hình Xception + Transformer Encoder

Hình 4.2 mô tả kiến trúc của mô hình thông qua các Mô-đun:

- Đầu vào (Video Frames): Video được biểu diễn thành chuỗi khung hình RGB với kích thước ($B \times T, 3, H, W$), trong đó B là batch, T là số khung hình, 3 là 3 kênh màu RGB, và (H, W) là kích thước không gian của ảnh.
- Xception: Từng khung hình được xử lý độc lập bởi mạng Xception để thu được các đặc trưng cấp cao. Và giữ nguyên thứ tự thời gian.
- Vector Feature ($B \times T, 2048$): Đầu ra là các vector đặc trưng ($B \times T, 2048$) biểu diễn thông tin không gian của mỗi frame.
- Reshape ($B, T, 2048$): Các đặc trưng được tái cấu trúc lại để phù hợp với đầu vào của mô-đun xử lý chuỗi thời gian.
- Transformer Encoder: Mô-đun này áp dụng cơ chế Multi-Head Attention để tính toán trọng số chú ý giữa tất cả các cặp khung hình trong chuỗi theo nhiều không gian con (head) khác nhau. Mỗi head học cách tập trung vào một khía cạnh quan hệ thời gian riêng, cho phép mô hình nắm bắt đồng thời nhiều loại phụ thuộc theo thời gian một cách linh hoạt và hiệu quả
- Fully Connected + Sigmoid: Đầu ra đại diện (đầu ra trung bình) từ Transformer Encoder được đưa qua 1 lớp fully connected và hàm Sigmoid để dự đoán xác suất trong bài toán phân loại nhị phân.

4.3.1. Trích xuất đặc trưng không gian bằng Xception

Tương tự với mô hình trên, dữ liệu đầu vào cũng sẽ được reshape từ (B, T, C, H, W) thành ($B \times T, C, H, W$) để đưa vào mạng Xception. Sau đó tiếp tục đưa dữ liệu đi qua Xception và reshape để thu được vector đặc trưng đầu ra với kích thước ($B, T, 2048$).

4.3.2. Mô hình hóa quan hệ thời gian bằng Transformer Encoder

Sau khi trích xuất được chuỗi các vector đặc trưng không gian từ mạng Xception và reshape, dữ liệu tiếp tục đi vào mạng Transformer Encoder. Mô hình này sử dụng cơ chế Multi-Head Self-Attention cho phép mỗi khung hình "quan sát" tất cả các khung hình khác trong cùng một video, điều này cho phép mô hình có thể quan sát những thay đổi bất thường ở các khung hình cách xa nhau.

Vector đặc trưng đầu ra của mô hình Xception có kích thước 2048 vì vậy Transformer Encoder cũng sẽ giảm kích thước xuống bằng với đầu ra để mô hình đảm bảo bảo toàn toàn bộ đặc trưng.

4.3.3. Tổng kết đặc trưng và phân loại

Sau khi qua mô hình Transformer Encoder, mô hình sẽ lấy trung bình đặc trưng tất cả các khung hình. Phương pháp này giúp nén toàn bộ thông tin quan trọng thu được từ Transformer thành một vector đại diện duy nhất cho cả video

Vector này sẽ tiếp tục đi qua 1 lớp fully connected để thực hiện tính toán giá trị logit. Cuối cùng sử dụng hàm Sigmoid để chuyển đổi logit thành xác suất, giúp phân video là fake/real.

4.4. Huấn luyện mô hình

Trước khi đi vào mô hình dữ liệu sẽ được xử lý để mô hình được học dữ liệu tốt nhất:

4.4.1. Trọng số cân bằng dữ liệu

Dữ liệu ban đầu có sự chênh lệch đáng kể giữa hai lớp real và fake, dẫn đến hiện tượng mất cân bằng dữ liệu. Do đó, trong quá trình huấn luyện, nhóm sử dụng Class Weighting (trọng số lớp). Trọng số này là pos_weight (positive class weight) được thiết lập bằng 0.45 nhằm cân bằng ảnh hưởng giữa hai lớp. Giá trị này được xác định là phù hợp nhất thông qua nhiều lần thử nghiệm huấn luyện và đánh giá hiệu năng mô hình.

Trọng số pos_weight được sử dụng trong hàm mất mát BCEWithLogitsLoss, giúp giảm thiên lệch của mô hình về lớp chiếm ưu thế và cải thiện khả năng học của lớp thiểu số.

4.4.2. Các siêu tham số huấn luyện mô hình

Các siêu tham số chính được sử dụng trong quá trình huấn luyện mô hình được trình bày như sau:

- Ảnh đầu vào được chuẩn hóa về kích thước 299×299 pixels đối với các mô hình Xception kết hợp LSTM và Xception kết hợp Transformer Encoder, trong khi mô hình EfficientNet kết hợp Temporal Attention sử dụng kích thước 224×224 pixels. Việc lựa chọn các kích thước này dựa trên kích thước tiêu chuẩn đầu vào của các mạng Xception, EfficientNet dùng cho trích xuất đặc trưng không gian.
- Batch size: 4, phù hợp với giới hạn bộ nhớ GPU khi xử lý dữ liệu video với thiết bị hiện tại.
- Learning rate: 5e-5

- Epoch: 40, số epoch này đủ để mô hình hội tụ trên tập huấn luyện mà không bị overfitting quá mức.

CHƯƠNG 5. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

5.1. Chỉ số đánh giá

Mô hình sẽ sử dụng 3 chỉ số chính để đánh giá hiệu suất của mô hình:

- Accuracy (Độ chính xác): Tỷ lệ đoán đúng mẫu real/fake trên tổng số mẫu.
- AUC-ROC: Khả năng phân biệt 2 lớp real và fake của mô hình.
- Confusion Matrix (Ma trận nhầm lẫn): Đánh giá chi tiết khả năng phân loại của mô hình để đưa ra đánh giá xem mô hình có bị thiên vị mẫu nào do sự mất cân bằng dữ liệu không ?

5.2. So sánh các mô hình

Bảng 5.1. So sánh kết quả giữa các mô hình

	Accuracy	AUC-ROC
EfficientNet + Temporal Attention	93,66%	97,68%
Xception + LSTM	92,8%	96,43%
Xception + Transformer Encoder	92,94%	97,09%

Thông qua Bảng 5.1, mô hình đạt kết quả tốt nhất là EfficientNet + Temporal Attention với Accracy đạt 93,66% và AUC-ROC đạt 97,68%. Mô hình này vượt trội hơn Xception + LSTM và Xception + Transformer Encoder về cả 2 chỉ số đánh giá. Điều này minh chứng rằng không phải mô hình càng phức tạp thì kết quả càng tốt.

Nguyên nhân có thể do mô hình phức tạp dễ bị overfitting hoặc yêu cầu lượng dữ liệu huấn luyện lớn hơn mà bộ dữ liệu hiện tại chưa đáp ứng đủ.

Kết hợp với cơ chế chú ý theo thời gian (Temporal Attention), mô hình có khả năng tập trung phân tích vào các vùng tín hiệu quan trọng nhất trong chuỗi video, thay vì xử lý mọi khung hình với mức độ ưu tiên như nhau. Điều này giúp mô hình đưa ra quyết định dựa trên bằng chứng mạnh mẽ và rõ ràng nhất, từ đó nâng cao độ tin cậy và hiệu suất tổng thể trong việc phát hiện video giả mạo

5.3. Nhận xét và đánh giá kết quả của mô hình

Mô hình EfficientNet + Temporal Attention đạt kết quả tốt nhất tại epoch thứ 23 với:

Bảng 5.2 Kết quả thực nghiệm

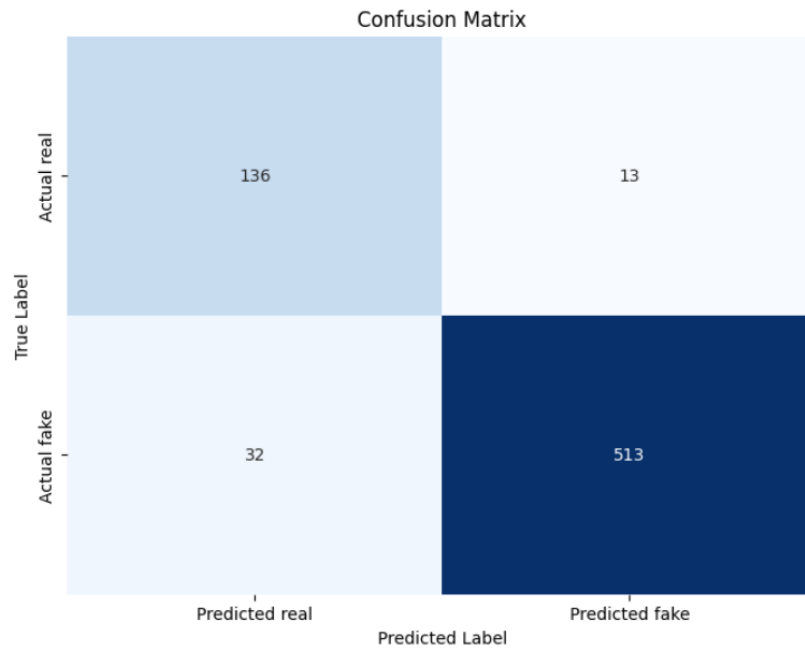
Chỉ số	Kết quả
Loss	0,02
Val acc	93,44%
Test acc	93,66%
AUC-ROC	97,68%

Thông qua thông số trên cho thấy:

- Độ chính xác của tập validation (val acc) và Độ chính xác của tập test (test acc) có kết quả khá sát nhau. Thậm chí đã hội tụ tại 1 điểm. Chứng tỏ mô hình không bị Overfitting (quá khớp) do đó mô hình có khả năng tổng quát hoá rất tốt khi gặp dữ liệu mới.
- Chỉ số AUC-ROC đạt kết quả khá cao (97,68%), cho thấy mô hình cực kì nhạy bén khi phân biệt 2 nhãn real và fake, cho thấy mô hình rất đáng tin cậy trong việc phát hiện video có sử dụng deepfake.
- Do đó mô hình cũng có thể được coi là phương pháp khả thi có tiềm năng có thể áp dụng được vào đời sống trong việc hỗ trợ phát hiện deepfake.

5.4. Phân tích ma trận nhầm lẫn

Do dữ liệu huấn luyện và kiểm tra có sự mất cân bằng lớn giữa hai lớp, việc chỉ dựa vào các chỉ số tổng hợp như Accuracy hay AUC-ROC là chưa đủ. Để đánh giá toàn diện và đảm bảo mô hình thực sự tốt, không thiên vị về lớp đa số (Fake), chúng tôi tiến hành phân tích chi tiết Ma trận Nhầm lẫn (Confusion Matrix) trên tập kiểm tra. Kết quả thu được như sau:



Ảnh 5.1. Ma trận nhầm lẫn

– Phân tích chỉ số cho thấy:

- + Khả năng nhận diện fake: Mô hình đã nhận diện chính xác 513/545 mẫu video giả, đạt tỷ lệ xấp xỉ 94,13%. Điều này cho thấy mô hình rất hiệu quả trong việc phát hiện các video deepfake, giúp giảm thiểu đáng kể nguy cơ lan truyền nội dung giả mạo gây hại.
- + Khả năng nhận diện real: Mô hình xác định đúng 136/149 video thật, đạt tỷ lệ 91,28%. Kết quả này ở mức rất cao đối với lớp thiểu số (chỉ chiếm ~21,5% dữ liệu tập kiểm thử). Điều này rất quan trọng trong các hệ thống thực tế, tránh các báo động giả gây hoang mang đối với người dùng.

– Đánh giá tổng quan:

Mặc dù tập dữ liệu có sự chênh lệch lớn về số lượng mẫu với đối tượng là khuôn mặt người (545 Fake so với 149 Real), ma trận nhầm lẫn cho thấy mô hình không bị thiên kiến hoàn toàn về lớp đa số. Việc kết hợp class weighting (trọng số lớp) trong quá trình huấn luyện đã giúp mô hình cân bằng tốt giữa việc phát hiện video giả và giữ lại video thật.

5.5. Thảo luận và ý nghĩa thực tiễn

Nghiên cứu này đem lại ý nghĩa thực tiễn cao cho bài toán phát hiện video giả mạo (Deepfake Detection). Với độ chính xác đạt 93,66%, mô hình thể hiện sự vượt trội đáng kể so với khả năng nhận diện trung bình của con người (68,7%) với cùng bộ dữ liệu [4]. Kết quả này mở ra tiềm năng ứng dụng rộng rãi trong nhiều lĩnh vực:

- An ninh mạng và Xác thực thông tin: Hỗ trợ các nền tảng mạng xã hội (Facebook, TikTok, YouTube) lọc và gắn nhãn cảnh báo cho nội dung giả mạo, chống lại các chiến dịch tin giả (fake news), tấn công lừa đảo (scam) hoặc bôi nhọ danh dự.
- Báo chí và Truyền thông: Giúp các cơ quan báo chí kiểm chứng tính xác thực của các video tư liệu trước khi đăng tải, đảm bảo tính chân thực của thông tin.
- Tư pháp và Pháp lý: Cung cấp công cụ hỗ trợ cho các cơ quan điều tra trong việc thẩm định bằng chứng video, phát hiện các bằng chứng bị giả mạo hoặc chỉnh sửa gian dối.
- Đời sống con người: Đặc biệt việc phổ biến các ứng dụng liên quan tới nhận diện deepfake giúp mọi người có thể tiếp cận và tự bảo vệ bản thân mình trước các cuộc gọi lừa đảo sử dụng deepfake vốn đang ngày càng tinh vi.

Nghiên cứu này không chỉ đóng góp về mặt kỹ thuật mà còn góp phần xây dựng một "tấm khiên" công nghệ đáng tin cậy, bảo vệ sự thật và an ninh trong kỷ nguyên số.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Thông qua đề tài này, nhóm đã nghiên cứu và xây dựng thành công các mô hình phát hiện deepfake với hiệu quả tương đối tốt. Tuy nhiên do hạn chế về thiết bị phần cứng cũng như chi phí đầu tư cho các phần mềm và tài nguyên huấn luyện nên vẫn còn nhiều hạn chế về dữ liệu và thời gian huấn luyện mô hình.

Mô hình phát hiện deepfake không chỉ phân biệt được ảnh thật hay giả mà còn hiểu quan hệ thời gian giữa các khung hình giúp mô hình phân tích, từ đó nâng cao khả năng phân tích và nhận diện video thật so với video được tạo bằng công nghệ deepfake.

Bên cạnh đó, trong quá trình thực hiện đề tài, nhóm đã vận dụng và mở rộng các kiến thức đã học như kỹ thuật tăng cường dữ liệu nhằm gia tăng tính đa dạng của tập huấn luyện, cũng như sử dụng trọng số cân bằng (pos_weight) để giảm thiểu hiện tượng mô hình thiên lệch do dữ liệu mất cân bằng,... Những biện pháp này đã góp phần cải thiện đáng kể hiệu suất và độ ổn định của mô hình.

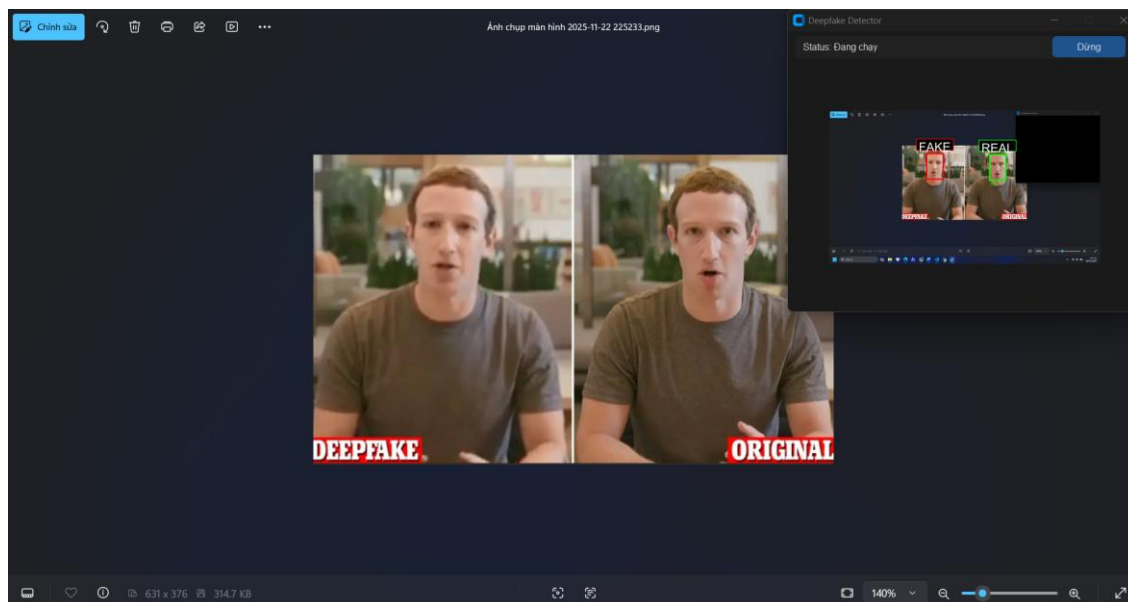
6.2. Hướng phát triển

Mặc dù kết quả của mô hình ở mức khả quan nhưng vẫn có thể cải thiện. Trong tương lai gần, nhóm đã có những dự định cụ thể như sau:

- Tăng cường số khung hình trích xuất được từ các video từ 10 lên 20, 30, hoặc hơn nữa.
- Ngoài dữ liệu RGB thì có thể thêm luồng dữ liệu ELA hay luồng trích xuất đặc trưng tần số khác.
- Huấn luyện và đánh giá mô hình trên các tập dữ liệu đa dạng hơn, không chỉ giới hạn ở FaceForensics++.
- Tiếp tục nghiên cứu, xây dựng và thực nghiệm thêm các kiến trúc mô hình khác nhằm nâng cao độ chính xác và khả năng tổng quát hóa của mô hình.

CHƯƠNG 7. DEMO ỨNG DỤNG

Ứng dụng được xây dựng bằng CustomTkinter, ý tưởng chính là sử dụng ứng dụng này quay màn hình của thiết bị sau đó phát hiện khuôn mặt, vẽ bounding box và phân loại real/fake đối với từng khuôn mặt.



Ảnh 7.1. Demo ứng dụng

Ứng dụng sử dụng CustomTkinter làm giao diện chính, hiện thị always-on-top với kích thước 480x380 pixels.

Để thu thập dữ liệu đầu vào, ứng dụng sử dụng thư viện mss để chụp màn hình liên tục. Cứ mỗi 10 khung hình mô hình nhận diện deepfake sẽ lấy 10 khung hình ấy để phân loại mỗi khuôn mặt độc lập, sau khi hoàn thành nhận diện, kết quả nhận diện được hiển thị trực tiếp trên giao diện, bao gồm vẽ bounding box và phân loại nhả real/fake tương ứng trên giao diện.

TÀI LIỆU THAM KHẢO

- [1] Tạp chí Sinh viên Khoa học Pháp luật, *Công nghệ Deepfake và những thách thức mới cho khung pháp lý chứng cứ điện tử tại Việt Nam*, 2024. Truy cập tại: <https://tapchisvkhplulaw.wordpress.com/2024/12/31/04-52-2024-cong-nghe-deepfake-va-nhung-thach-thuc-moi-cho-khung-phap-ly-chung-cu-dien-tu-tai-viet-nam/>
- [2] VTC News, “*Báo cáo sốc: Việt Nam đứng đầu gia tăng gian lận deepfake, xếp trên Nhật Bản*”. Truy cập tại: [Báo cáo sốc: Việt Nam đứng đầu gia tăng gian lận deepfake, xếp trên Nhật Bản](#)
- [3] GVF, *Deepfake và những nguy cơ với an toàn thông tin tại Việt Nam*. Truy cập tại: <https://www.ugvf.org/vi/deepfake-va-nhung-nguy-co-voi-an-toan-thong-tin-tai-viet-nam/>
- [4] A. Rössler et al., *FaceForensics++: Learning to Detect Manipulated Facial Image*. Truy cập tại: <https://arxiv.org/pdf/1901.08971>
- [5] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. Truy cập tại: <https://arxiv.org/pdf/1912.01703>
- [6] Lawson, A., Hendrick, S., Rausch, N., & Sica, J., et al. (2024). *Shaping the Future of Generative AI: The Impact of Open Source Innovation*. The Linux Foundation Research. Truy cập tại: https://www.linuxfoundation.org/hubfs/LF%20Research/lfr_genai24_111924.pdf
- [7] GeeksforGeeks, *Transformers in Machine Learning*. Truy cập tại: <https://www.geeksforgeeks.org/machine-learning/getting-started-with-transformers/>
- [8] A. Vaswani et al., "Attention is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. Truy cập tại: <https://arxiv.org/pdf/1706.03762>
- [9] GeeksforGeeks, *Xception Neural Network Architecture*. Truy cập tại: <https://www.geeksforgeeks.org/computer-vision/xception/>

- [10] MTCNN Documentation, *Introduction to MTCNN*. Truy cập tại: <https://mtcnn.readthedocs.io/en/latest/introduction/>
- [11] GeeksforGeeks, “*What is LSTM - Long Short Term Memory?*” Truy cập tại: <https://www.geeksforgeeks.org/deep-learning/deep-learning-introduction-to-long-short-term-memory/>
- [12] GeeksforGeeks, "EfficientNet Architecture," *GeeksforGeeks*, 2023. Truy cập tại: <https://www.geeksforgeeks.org/computer-vision/efficientnet-architecture/>
- [13] Ultralytics, “*What is EfficientNet? A quick overview*”. Truy cập tại: [EfficientNet: Compound Scaling & Architecture | Ultralytics](#)
- [14] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019. Truy cập tại: [EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks](#)
- [15] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018. Truy cập tại: <https://elibrary.pearson.de/book/99.150005/9781292223070>
- [16] L. Zhang, X. Zhang, và các cộng sự, "STA-TSN: Spatial-Temporal Attention Temporal Segment Network for action recognition in video," *PLOS ONE*, vol. 14, no. 11, pp. e0224175, 2019. DOI: 10.1371/journal.pone.0224175. Truy cập tại: <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0224175>